

넘파이 배열

생성 · 구조 · 추출

리스트를 배열로 바꾸는 순간부터 — 반복문 없이 수백만 개를 한 줄로 계산한다

10_01 배열 생성과 구조 · 10_02 데이터 추출과 통계 분석 PART 01

핵심 키워드

가상환경 · pip install · ndarray · np.array · np.arange · np.linspace · np.zeros · np.full
.ndim · .shape · .size · .dtype · .astype() · .reshape() · .flatten()
a[0] · a[-1] · a[1:4] · a[:,2] · data[0, 1] · data[:, 0]

2026년 8월 10일 (월) · 15일차 · 37~38차시

국립금오공과대학교 컴퓨터공학부 박민호

이 책을 읽는 법

뱃지 · 코드블록 · 지난 회차와의 연결

1. 항목 뱃지 — 무엇을 배우는 중인지 한눈에

모든 항목 제목 앞에는 색 뱃지가 붙는다. 뱃지는 그 항목이 **문법상 무엇인가**를 알려준다. 이번 회차는 넘파이 첫 시간이라 함수·속성·메서드가 한 화면에 뒤섞여 나온다. 셋을 구분하지 못하면 오늘 배운 것을 전부 "넘파이 명령어"라는 한 덩어리로 기억하게 되고, 시험이나 면접에서 반드시 무너진다.

뱃지	무엇인가	이번 회차 예
함수	이름 뒤에 괄호. 앞에 점(.)이 없다	<code>np.array()</code> · <code>np.arange()</code> · <code>np.linspace()</code>
메서드	값 뒤에 점을 찍고 부르는 함수. 괄호 있음	<code>.astype()</code> · <code>.reshape()</code> · <code>.flatten()</code>
속성	값 뒤에 점을 찍지만 괄호가 없음	<code>.ndim</code> · <code>.shape</code> · <code>.size</code> · <code>.dtype</code>
자료형	값의 종류 자체	<code>ndarray</code> · <code>int64</code> · <code>float64</code>
문법	키워드가 아니라 표기 규칙	<code>a[1:4]</code> · <code>data[:, 0]</code>
명령어	파이썬이 아니라 터미널에서 치는 것	<code>python -m venv</code> · <code>pip install</code>
개념	문법이 아닌 사고방식·배경 지식	벡터 연산 · 차원 · 브로드캐스팅의 씨앗

2. 코드블록 두 종류 — 파이썬인가, 터미널인가

오늘은 처음으로 터미널 명령을 친 날이다. 그래서 이 책에는 코드블록이 두 가지 색으로 나온다. 색만 보고도 "이건 .py 파일에 쓰는 것"과 "이건 검은 창에 치는 것"을 구분할 수 있어야 한다. 이 둘을 섞으면 `pip install numpy`를 파이썬 파일에 적어 놓고 왜 안 되냐고 묻게 된다.

파이썬 · 파란 바 · .py 파일에 쓴다

```
import numpy as np
temp = np.array([70, 72, 71])
print(temp)
```

▶ [70 72 71]

TERMINAL · 회색 바 · 검은 창에 친다

```
python -m venv .venv
source .venv/bin/activate
pip install numpy
```

▶ Successfully installed numpy-2.2.6

3. 지난 회차와의 연결 — 9주간 배운 것이 여기서 만난다

넘파이는 갑자기 등장한 새 주제가 아니다. 지금까지 배운 파이썬 문법 위에 얹히는 도구다. 아래 표의 왼쪽을 이미 알고 있다면 오른쪽은 대응만 시키면 된다.

이미 배운 것	오늘 만나는 것	달라지는 지점
list 리스트	ndarray 배열	섞어 담기 불가 · 대신 계산이 빠름
lst * 3 → 반복	arr * 3 → 곱셈	같은 연산자가 전혀 다른 뜻
range(0, 10, 2)	np.arange(0, 10, 2)	range는 게으른 객체, arange는 실제 배열
lst[1:4] 슬라이싱	arr[1:4] 슬라이싱	규칙은 동일 — 시작 포함, 끝 제외
lst[0][1] 중첩 대괄호	arr[0, 1] 쉼표 표기	넘파이는 쉼표 표기를 정식으로 씀
import 모듈 (12일차)	import numpy as np	외부 라이브러리 · 별명(as) 붙이기
for 반복문으로 전체 계산	배열 한 줄로 전체 계산	"벡터 연산" — 다음 시간의 핵심

주의

이번 회차 자료 결손 고지. 수업 전사본(STT)이 마지막 부분에서 끊겨 있다(935줄, "이걸 진행할"에서 중단). 2차원 슬라이싱 `data[:, 0]`을 실제로 타이핑하며 설명한 구간이 전사본에 남아 있지 않다. 이 부분은 교안 10_02 p11~p16과 강사 배포 코드 10_02_04_numpy_2d_slice.py로 교차 복원했으므로 내용 자체는 정확하다. 다만 강사가 그 자리에서 덧붙인 구두 설명이 있었다면 이 책에는 빠져 있을 수 있다.

연결

오늘 진도 범위. 교안 10_01 「배열 생성과 구조」는 50쪽 전 범위, 교안 10_02 「데이터 추출과 통계 분석」은 62쪽 중 PART 01(인덱싱·슬라이싱, p4~p16)까지다. PART 02 배열 연산·필터링과 PART 03 기초 통계는 진도 밖이므로 부록 C의 「다음 시간 예고」로 분리했다.

CONTENTS

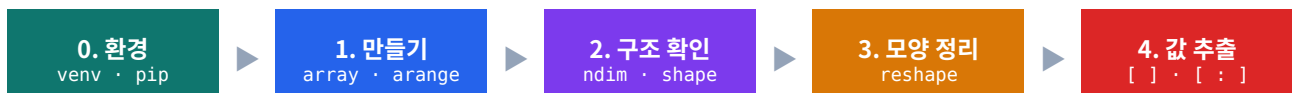
목차

오늘 배운 것 전체 지도

장 · STEP	무엇을 배우나	핵심
CHAPTER 0	오늘의 지도 — 왜 넘파이인가, 함수·메서드·속성 3초 판별법	<code>ndarray</code> · 벡터 연산
STEP 1	가상환경과 설치 — 넘파일을 쓰기 전에 반드시 거치는 관문	<code>venv</code> · <code>pip install</code>
STEP 2	리스트의 한계와 배열의 등장 — 곱셈이 반복이 되는 문제	<code>np.array()</code>
STEP 3	배열을 만드는 네 가지 함수 — 값·간격·개수·초기화	<code>arange</code> · <code>linspace</code> · <code>zeros</code>
STEP 4	배열의 구조를 읽는 네 속성 — 데이터를 만나면 가장 먼저	<code>.ndim</code> · <code>.shape</code> · <code>.size</code> · <code>.dtype</code>
STEP 5	모양 바꾸기 — 값은 그대로, 배치만 바꾸기	<code>.reshape()</code> · <code>.flatten()</code>
STEP 6	값 꺼내기(인덱싱) — 번호 하나로 콧 집기, 1·2차원	<code>a[0]</code> · <code>data[0, 1]</code>
STEP 7	구간 꺼내기(슬라이싱) — 콜론이 전부를 의미한다	<code>a[1:4]</code> · <code>data[:, 0]</code>
부록 A	치트시트 — 오늘 나온 함수·속성·메서드 전체 요약	한 장 요약
부록 B	오류·함정 사전 + 제출 코드 교정 — 화씨 변환 버그 포함	$\times \rightarrow \checkmark$
부록 C	실습 하이라이트 · 미작성 실습 보충답안 · 다음 시간 예고	실습 1~8

오늘 하루의 흐름 — 다섯 단계

강의는 「만들기 → 구조 확인 → 모양 정리 → 값 추출」이라는 하나의 흐름으로 짜여 있다. 교안 10_01의 마지막 장(p50)과 10_02의 첫 장이 정확히 이 순서로 이어진다. 날개 함수를 외우려 하지 말고 **이 다섯 칸 중 어디에 쓰는 도구인지**로 묶어서 기억하는 편이 훨씬 오래 간다.



설비 적용

교안 10_02 p3은 이번 회차의 배경 공정을 **CNC 가공(MCT)** 으로 잡았다. 공구를 회전시켜 금속 덩어리를 깎아 형상을 만드는 정밀가공 설비로, **스핀들 회전수(rpm) · 토크 · 온도**를 기록한다. 앞으로 나오는 예제 배열 rpm, torque가 전부 이 설비에서 나온 값이라고 생각하면 된다. 강사는 노트북 알루미늄 유니바디를 깎는 공정을 예로 들었다 — 통 알루미늄을 레이저와 절삭공구로 깎아내는 그 기계가 MCT다.

오늘 나간 진도 — 교안 어디까지인가

교안은 두 권이 배포됐지만 **두 번째 교안은 앞부분만** 나갔다. 복습할 때 어디까지가 오늘 범위인지 헷갈리지 않도록 정리해 둔다.

교안	쪽수	오늘 나간 범위	이 책에서
10_01 배열 생성과 구조	50쪽	전 범위 (실습 1~8 포함)	STEP 2~5
10_02 데이터 추출과 통계 분석	62쪽	PART 01 인덱싱·슬라이싱 (p4~p16)	STEP 6~7
└ PART 02 배열 연산·필터링	p17~p38	진도 밖	부록 C 예고
└ PART 03 기초 통계	p39~p62	진도 밖	부록 C 예고
보조 교재 4종 (guide · math guide)	11쪽	배포만, 수업 중 언급	참고 자료

주의

교안 10_01 파일명이 ..._36_260807_2.pdf라 **8월 7일 자료로 보이지만 진도는 오늘 나갔다**. 강사가 미리 올려 둔 것이다. 파일 날짜가 아니라 **수업에서 실제로 다룬 시점**을 기준으로 복습 순서를 잡자.

오늘의 지도

왜 넘파이인가 · 함수와 속성을 가르는 3초 판별법

0-1. 왜 하필 넘파이인가 — 하루 수백만 개의 숫자

공장 설비 한 대에는 온도·진동·압력 센서가 붙어 **1초에도 여러 번** 값을 쏟아낸다. 설비가 수십 대면 하루에 수백만 개의 숫자가 쌓인다. 사람이 직접 계산할 수 없고, 파이썬 리스트와 `for` 반복문으로도 감당이 안 된다. 그래서 등장하는 것이 넘파이다.

중요한 건 넘파이가 **끝이 아니라 시작**이라는 점이다. 앞으로 배울 Pandas도, 그래프를 그리는 Matplotlib도 전부 넘파이 위에서 돌아간다. 오늘 배열을 이해해 두지 않으면 다음 달 Pandas 수업에서 `df.values`가 무엇인지, `df.shape`이 왜 튜플인지 전혀 감을 못 잡게 된다.

도구	역할	넘파이와의 관계
NumPy	숫자 묶음 계산 — 배열 하나를 통째로 연산	가장 기초. 나머지 전부의 토대
Pandas	표 데이터 처리 — 열 이름·행 이름이 붙은 표	내부가 넘파이 배열. 다음 주 진도
Matplotlib	그래프 시각화 — 꺾은선·산점도·히스토그램	x·y축 데이터를 넘파이 배열로 받음
scikit-learn	머신러닝 모델 학습·예측	입력·출력이 전부 넘파이 배열

비유

리스트가 **아무거나 담을 수 있는 장바구니**라면, 배열은 **같은 크기 계란만 들어가는 계란판**이다. 계란판은 아무거나 못 담는 대신, 몇 번째 칸에 무엇이 있는지 계산 없이 바로 찾아낼 수 있다. 넘파이가 빠른 이유가 정확히 이것이다 — 제약을 받아들인 대가로 속도를 얻는다.

0-2. 함수 · 메서드 · 속성 — 3초 판별법

오늘 나온 표기법을 그대로 늘어놓아 보자. `np.array()`, `np.arange()`, `arr.ndim`, `arr.shape`, `arr.astype()`, `arr.reshape()`. 뒤죽박죽으로 보이지만 **점(.)과 괄호()** 두 가지만 보면 3초 안에 갈린다. 교안 10_01 p28이 이 구분에 슬라이드 한 장을 통째로 쓴 이유가 있다.

질문	판정	예
① 앞에 점(.)이 있나? → 없다	함수 독립 함수. 모듈 이름이 앞에 온다	<code>np.array([1,2])</code>

질문	판정	예
② 점이 있고, 뒤에 괄호가 있다	메서드 값에게 시키는 동작	<code>arr.reshape(2,3)</code>
③ 점이 있고, 뒤에 괄호가 없다	속성 값이 이미 가진 정보를 꺼내 봄	<code>arr.shape</code>

교안의 표현을 빌리면 이렇다 — "무언가를 시키는 명령(함수·메서드)은 괄호가 붙고, 이미 가진 정보를 꺼내 보는 속성은 괄호가 없다." 차원·형태·크기 같은 정보는 배열이 만들어질 때 이미 달려 있어서 새로 계산하는 게 아니라 그냥 들여다보는 것이다.

괄호 하나로 갈리는 결과

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

# 속성 — 괄호 없음. 붙이면 오류
print(arr.shape)
# print(arr.shape()) ← TypeError: tuple is not callable

# 메서드 — 괄호 필수. 빼면 함수 객체가 그대로 찍힌다
print(arr.flatten())
print(arr.flatten)

▶ (2, 3)
▶ [1 2 3 4 5 6]
▶ <built-in method flatten of numpy.ndarray object at 0x7f...>
```

자주 하는 실수

`arr.shape()`처럼 속성에 괄호를 붙이면 `TypeError: 'tuple' object is not callable`이 뜬다. 반대로 `arr.flatten`처럼 메서드에서 괄호를 빼면 **오류가 안 나고** 이상한 문자열이 찍힌다. 후자가 훨씬 위험하다 — 오류가 안 나니 그냥 넘어가고, 나중에 결과가 틀린 이유를 못 찾는다.

0-3. 오늘 나온 이름 전체 — 분류부터 해 두고 시작

본문에 들어가기 전에 오늘 등장할 이름을 분류해 둔다. 이 표를 먼저 훑어 두면, 각 STEP에서 "아, 이건 아까 속성 칸에 있던 거구나" 하고 자리를 잡을 수 있다.

분류	이름	한 줄
명령어	<code>python -m venv .venv</code>	현재 폴더에 가상환경을 만든다 (터미널)
명령어	<code>pip install numpy</code>	가상환경 안에 넘파이를 설치한다 (터미널)

분류	이름	한 줄
키워드	<code>import ... as ...</code>	라이브러리를 별명으로 불러온다
함수	<code>np.array(리스트)</code>	값을 이미 알 때 — 리스트를 배열로
함수	<code>np.arange(시작, 끝, 간격)</code>	간격이 중요할 때 — 끝값 제외
함수	<code>np.linspace(시작, 끝, 개수)</code>	개수가 중요할 때 — 끝값 포함
함수	<code>np.zeros(n) / np.ones(n)</code>	0 또는 1로 채운 빈 그릇
함수	<code>np.full(n, 값)</code>	지정한 값으로 채움
속성	<code>.ndim</code>	차원 개수 (1차원? 2차원?)
속성	<code>.shape</code>	행·열 크기 — 튜플로 나옴
속성	<code>.size</code>	값의 총 개수 — shape을 곱한 값
속성	<code>.dtype</code>	자료형 — int64 / float64
메서드	<code>.astype(자료형)</code>	자료형 변환 — 소수점은 버림
메서드	<code>.reshape(행, 열)</code>	모양 변경 — size는 보존
메서드	<code>.flatten()</code>	2차원을 1차원 한 줄로 펴
문법	<code>a[0] · a[-1]</code>	인덱싱 — 값 하나 꺼내기
문법	<code>a[1:4] · a[:, 2]</code>	슬라이싱 — 구간·간격 꺼내기
문법	<code>data[0, 1] · data[:, 0]</code>	2차원 — 행·열 지정
자료형	<code>ndarray</code>	넘파이의 배열 자료형

STEP 1

가상환경과 설치

넘파이를 쓰기 전에 반드시 거치는 관문

오늘 수업은 코드가 아니라 **검은 터미널 창**에서 시작했다. 지금까지 배운 문법은 파이썬을 설치하면 전부 따라오는 것들이었지만, 넘파이는 다르다. **외부 라이브러리**라서 따로 받아야 한다. 그런데 그냥 받으려 하면 거절당한다. 그 이유와 해법이 STEP 1의 전부다.

1-1. 외부 라이브러리를 찾는 곳 — PyPI

개념 PyPI (pypi.org) 파이썬 라이브러리 공식 창고

정의 전 세계 개발자가 만든 파이썬 패키지가 모여 있는 저장소. 넘파이·판다스를 포함해 60만 개 이상이 올라와 있다.

왜 쓰나 라이브러리를 쓰기 전에 **먼저 검색부터** 하는 습관이 필요하다. 이름이 비슷한 가짜 패키지(타이포스쿼팅)가 실제로 존재하고, 설치 순간 악성코드가 실행되는 사고가 매년 보고된다. 정확한 철자·최신 버전·의존 라이브러리를 pypi.org에서 확인한 뒤 설치하는 것이 실무 기본기다.

응용 현업에서는 사내 **프라이빗 미러**(Nexus, Artifactory)를 두고 승인된 패키지만 받게 하는 경우가 많다. 제조 DX 조직에 들어가면 십중팔구 pip.conf에 사내 index-url이 박혀 있는 환경을 만나게 된다. 그때 "pip은 pypi.org에서 받는다"만 알고 있으면 왜 회사 망에서 설치가 되는지 이해하지 못한다.

```
# 설치 전 확인 습관
# 1) pypi.org 에서 철자·최신 버전 확인
# 2) 설치 후 버전 검증
pip show numpy
-----
> Name: numpy
> Version: 2.2.6
> Location: /home/user/project/.venv/lib/python3.12/site-packages
```

명령어 pip 파이썬 라이브러리 관리 매니저

정의 PyPI에서 패키지를 내려받아 설치·삭제·조회하는 도구. 파이썬을 설치하면 함께 깔린다.

형태 pip install 패키지명 / pip list / pip show 패키지명 / pip uninstall 패키지명

TERMINAL

```
pip install numpy
pip list
pip uninstall numpy
```

- ▶ Successfully installed numpy-2.2.6
- ▶ numpy 2.2.6
- ▶ Successfully uninstalled numpy-2.2.6

왜 쓰나 설치를 손으로 파일 복사해서 하는 시대는 끝났다. pip 한 줄이면 의존 라이브러리까지 알아서 따라온다.

응용 실무에서는 설치한 목록을 **파일로 고정**해 팀원과 공유한다. `pip freeze > requirements.txt`로 버전을 통째로 박제하고, 새 팀원은 `pip install -r requirements.txt` 한 줄로 똑같은 환경을 만든다. 포트폴리오 레포에 이 파일이 없으면 "실행이 안 되는 프로젝트"가 된다 — 면접에서 감점 요소다.

```
pip freeze > requirements.txt
cat requirements.txt
```

```
# 다른 사람 PC에서
pip install -r requirements.txt
```

- ▶ numpy==2.2.6

주의

pip은 파이썬 **밖**의 명령이다. .py 파일 안에 `pip install numpy`라고 적으면 `SyntaxError`가 난다. 반드시 터미널에 친다.

1-2. 왜 그냥 설치하면 거절당하는가

터미널을 열고 곧바로 `pip install numpy`를 치면, 최근 파이썬 환경(특히 리눅스·macOS)에서는 설치가 **거절된다**. 시스템 전체에 영향을 주는 설치이기 때문이다. 강사가 강조한 부분이 정확히 여기서다.

TERMINAL — 거절당하는 순간

```
pip install numpy
```

- ▶ error: externally-managed-environment
- ▶
- ▶ × This environment is externally managed
- ▶ ↳ To install Python packages system-wide, try apt install ...
- ▶ If you wish to install a non-Debian-packaged Python package,
- ▶ create a virtual environment using python3 -m venv path/to/venv

개념

왜 막아 두었나. 프로젝트 A는 넘파이 1.24가 필요하고, 프로젝트 B는 2.2가 필요할 수 있다. 시스템 전체에 하나만 깔면 둘 중 하나는 반드시 깨진다. 게다가 운영체제 자체가 파이썬 패키지를 쓰고 있어서, 시스템 패키지를 덮어쓰면 OS 도구가 망가진다. 그래서 **"작업 디렉터리마다 별도의 환경을 만들고 그 안에 설치하라"**는 것이 현재의 표준이다.

1-3. 가상환경 — 프로젝트마다 독립된 방 만들기

명령어 `python -m venv .venv` 현재 폴더에 가상환경 생성

정의 `venv` 모듈을 실행해 현재 경로에 독립된 파이썬 환경 폴더를 만든다. 관례상 폴더 이름은 `.venv`.

형태 `python -m venv` 폴더명

TERMINAL

```
# 1. 현재 경로에 가상환경 생성
python -m venv .venv

# 2. 가상환경 활성화 (macOS / Linux)
source .venv/bin/activate

# 3. 작업·실행이 끝나면 종료
deactivate
```

▶ `(.venv) $` ← 프롬프트 앞에 `(.venv)` 가 붙으면 활성화 성공

왜 쓰나 프로젝트마다 필요한 라이브러리와 버전이 다르기 때문이다. `.venv` 안에 설치하면 그 폴더를 지우는 것만으로 환경이 깨끗하게 정리된다. 시스템은 전혀 건드리지 않는다.

응용 윈도우는 활성화 명령이 다르다. PowerShell에서는 `.venv\Scripts\Activate.ps1`, cmd에서는 `.venv\Scripts\activate.bat`이다. 그리고 `.venv` 폴더는 절대 Git에 올리지 않는다 — **용량이 수백 MB이고 OS 마다 내용이 다르다.** `.gitignore`에 반드시 넣는다.

```
# Windows PowerShell
.venv\Scripts\Activate.ps1

# Windows cmd
.venv\Scripts\activate.bat

# .gitignore 에 추가할 것
# .venv/
```

▶ `(.venv) PS C:\project>`

개념

-m은 "모듈을 스크립트처럼 실행하라"는 뜻이다. 12일차에 배운 import와 같은 모듈 개념이 여기서 명령줄로 튀어나온 것이다.

주의

활성화를 잊고 `pip install numpy`를 치면 시스템에 설치를 시도해 다시 거절당하거나, 되더라도 프로젝트와 무관한 곳에 깔린다. **프롬프트 앞에 (.venv)가 보이는지 매번 눈으로 확인하는 습관을 들이자. 터미널을 새로 열 때마다 활성화는 다시 해야 한다.**

설비 적용

제조 현장 분석 서버는 인터넷이 막혀 있는 경우가 대부분이다. 이때는 인터넷 되는 PC에서 `pip download numpy -d ./pkgs`로 파일을 통째로 받아 USB로 옮긴 뒤, 현장에서 `pip install --no-index --find-links=./pkgs numpy`로 설치한다. **폐쇄망 설치**는 제조 DX 직군 면접에서 실제로 나오는 질문이다.

1-4. 불러오기 — import numpy as np

키워드 import numpy as np 넘파일을 np라는 별명으로 불러오기

정의 import는 라이브러리를 가져온다는 뜻, as np는 앞으로 np라는 짧은 별명으로 부르겠다는 뜻.

형태 import 라이브러리이름 as 별명

```
import numpy as np

print(np.array([1, 2, 3]))

▶ [1 2 3]
```

왜 쓰나 `numpy.array(...)`라고 매번 쓰면 코드가 길어진다. np는 전 세계 분석가가 똑같이 쓰는 약속이다. 남의 코드를 읽을 때도, 내 코드를 남이 읽을 때도 이 약속 덕분에 설명이 필요 없다.

응용 별명 약속은 넘파이만의 것이 아니다. 데이터 분석 스택 전체가 같은 관례를 따른다. 새 분석 파일을 만들 때 아래 세 줄부터 치고 시작하는 습관을 들이면 좋다.

```
import numpy as np          # 배열 계산
import pandas as pd         # 표 데이터
import matplotlib.pyplot as plt # 그래프

# 이 세 줄이 데이터 분석 파일의 표준 도입부
```

자주 하는 실수

`import numpy as np`를 빼먹고 `np.array(...)`를 쓰면 `NameError: name 'np' is not defined`가 뜬다. 파일마다 맨 위에 반드시 한 번 적는다.

1-5. 설치 확인과 흔한 문제 해결

가상환경을 켜고 설치했는데도 `ModuleNotFoundError`가 뜨는 상황이 가장 자주 생긴다. 원인은 대부분 **설치한 파이썬과 실행한 파이썬이 다른 것**이다.

TERMINAL — 경로가 `.venv` 안을 가리켜야 정상

```
# 지금 어떤 파이썬을 쓰고 있는지 확인
which python
```

```
# 넘파이가 실제로 어디에 깔렸는지
pip show numpy
```

```
▶ /home/user/project/.venv/bin/python
▶ Location: /home/user/project/.venv/lib/python3.12/site-packages
```

증상	원인	해결
<code>ModuleNotFoundError: numpy</code>	가상환경을 켜지 않고 실행	활성화 후 다시 실행
설치는 됐는데 import 실패	에디터가 시스템 파이썬을 씀	VS Code 우하단 인터프리터를 <code>.venv</code> 로 변경
<code>externally-managed-environment</code>	시스템 전체에 설치 시도	가상환경 만들고 그 안에 설치

주의

VS Code를 쓴다면 **우하단 인터프리터 표시**에 `.venv`가 붙어 있어야 한다. 없으면 `Ctrl+Shift+P → Python: Select Interpreter`에서 `.venv` 안의 파이썬을 고른다. 이 설정 하나가 앞으로 겪을 import 오류의 절반을 없앤다.

STEP 2

리스트의 한계와 배열의 등장

곱셈이 곱셈이 아니게 되는 문제

2-1. 리스트에 3을 곱하면 무슨 일이 일어나는가

강의는 넘파이 문법이 아니라 **리스트가 안 되는 장면**부터 보여 주며 시작했다. 이 장면을 직접 눈으로 봐야 넘파이가 왜 필요한지 몸으로 이해된다.

리스트 — 곱셈이 반복이 된다

```
numbers = [1, 2, 3, 4, 5]
```

```
# 리스트에 3을 곱하면?
```

```
print(numbers * 3)
```

```
# 각 값을 3배 하려면 반복문이 필요하다
```

```
result = []
```

```
for n in numbers:
```

```
    result.append(n * 3)
```

```
print(result)
```

```
▶ [1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

```
▶ [3, 6, 9, 12, 15]
```

`numbers * 3`이 **계산이 아니라 반복**이 됐다. 리스트에게 `*`는 "이어 붙여라"라는 뜻이기 때문이다. 각 값에 3을 곱하고 싶으면 반복문을 돌려야 한다. 값이 다섯 개면 괜찮지만, 센서 데이터 100만 개에 이걸 하면 파이썬은 100만 번 반복문을 돌면서 눈에 띄게 느려진다.

항목	리스트 (list)	배열 (ndarray)
자료형	서로 다른 자료형 혼용 가능 (글자·숫자·리스트)	동일한 자료형만 저장 가능 (통일됨)
전체 계산	반복문이 필요함	한 줄로 처리
처리 속도	느림 — 값마다 파이썬이 개입	빠름 — C 수준에서 한 번에
통계 계산	직접 합계·개수 세서 나눠야 함	함수 하나 (다음 시간)
메모리	값마다 흩어져 저장	한 덩어리로 차곡차곡 정렬
* 연산자	이어 붙이기 (반복)	각 값에 곱하기 (계산)

2-2. ndarray — 넘파이가 데이터를 담는 그릇

자료형 ndarray 넘파이의 배열 자료형

정의 N-dimensional array의 줄임말. 같은 자료형의 값만 담고, 정해진 형태(한 줄 또는 표 모양)를 가진다.

```
import numpy as np

numbers = [1, 2, 3, 4, 5]
np_numbers = np.array(numbers)

print(np_numbers)
print(type(np_numbers))

# 배열에 3을 곱하면 — 이번엔 진짜 곱셈
print(np_numbers * 3)
```

```
▶ [1 2 3 4 5]
▶ <class 'numpy.ndarray'>
▶ [ 3  6  9 12 15]
```

왜 쓰나 배열이 같은 자료형만 담는 덕분에 메모리에 차곡차곡 정렬되고, 그래서 계산이 빠르다. 제약처럼 보이지만 속도의 비결이다.

응용 출력 형태만 봐도 리스트인지 배열인지 구분할 수 있다. 리스트는 **쉼표로 값을 구분하고**, 배열은 **공백으로 구분한다**.

디버깅 중 print만 찍어 놓고 "이게 리스트였나 배열이었나" 헛갈릴 때 이 차이 하나로 즉시 판별된다.

```
print([1, 2, 3])          # 리스트
print(np.array([1, 2, 3])) # 배열

# 확실히 하려면 type() 으로
print(type([1, 2, 3]).__name__)
print(type(np.array([1, 2, 3])).__name__)
```

```
▶ [1, 2, 3]
▶ [1 2 3]
▶ list
▶ ndarray
```

개념

배열의 두 가지 핵심 특징 — ① 같은 자료형만 담는다 ② 정해진 형태를 가진다. 이 둘이 리스트와의 모든 차이를 만든다.

함수 np.array() 리스트를 배열로 변환

정의 리스트를 넣으면 넘파이 배열(ndarray)로 만들어 준다. 값을 이미 알고 있을 때 쓰는 가장 기본 함수.

형태 np.array(리스트)

```
temp = np.array([70.5, 69.8, 73.7])
print(temp)
```

```
# 묶음 계산 — 반복문 없이 전체에 적용
print(temp + 5)
print(temp * 2)
```

```
▶ [70.5 69.8 73.7]
▶ [75.5 74.8 78.7]
▶ [141. 139.6 147.4]
```

왜 쓰나 temp + 5 한 줄로 모든 값에 5가 더해진다. 반복문이 전혀 없다. 이것이 다음 시간에 배울 **벡터 연산**의 첫 맛보기다.

응용 2차원 배열을 만들려면 **리스트 안에 리스트**를 넣는다. 설비 데이터는 대부분 「행=시점, 열=센서」인 2차원 표라서, 실무에서는 이 형태를 훨씬 자주 쓴다.

```
# 행=시점, 열=센서 (온도, 진동)
sensor = np.array([
    [70.5, 2.1],
    [69.8, 2.3],
    [73.7, 2.0]
])
print(sensor)
print(sensor.shape)
```

```
▶ [[70.5 2.1]
   [69.8 2.3]
   [73.7 2. ]]
▶ (3, 2)
```

주의

np.array(1, 2, 3)처럼 쓰면 오류다. 반드시 **리스트 하나로 묶어서** np.array([1, 2, 3])으로 전달해야 한다.

2-3. 자료형은 자동으로 정해진다 — 하나라도 소수점이면 전부 실수

강사가 실습 중 오타를 내며까지 강조한 규칙이 있다. **소수점이 하나라도 섞이면 배열 전체가 실수(float)가 된다**. 배열은 같은 자료형만 담을 수 있으므로, 더 넓게 담을 수 있는 실수 쪽으로 자동 통일하는 것이다.

자동 자료형 결정 규칙

```
a = np.array([1, 2, 3])          # 전부 정수
b = np.array([1, 2, 3.0])        # 하나만 실수
c = np.array([70, 2.1])          # 하나만 실수

print(a, a.dtype)
print(b, b.dtype)
print(c, c.dtype)
```

```
▶ [1 2 3] int64
▶ [1. 2. 3.] float64
▶ [70.  2.1] float64
```

주의

출력에서 70.처럼 **점만 찍히고 뒤가 비어 있으면 실수 배열**이라는 신호다. [70. 2.1]을 보고 "70은 정수네" 하고 넘어가면 안 된다. 만든 배열의 dtype이 예상과 다를 때 이 규칙을 가장 먼저 떠올리자.

2-4. 배열이 빠른 이유 — 실제로 재 보기

"배열이 빠르다"는 말은 여러 번 들었지만, 얼마나 빠른지 숫자로 본 적은 없을 것이다. 100만 개를 두 배로 만드는 작업을 리스트와 배열로 각각 재 보면 차이가 분명해진다.

같은 일, 다른 속도

```
import numpy as np
import time

N = 1_000_000
lst = list(range(N))
arr = np.arange(N)

t = time.perf_counter()
lst2 = [x * 2 for x in lst]      # 리스트 — 100만 번 반복
print(f"리스트: {time.perf_counter() - t:.4f}초")

t = time.perf_counter()
arr2 = arr * 2                  # 배열 — 한 줄
print(f"배열 : {time.perf_counter() - t:.4f}초")
```

```
▶ 리스트: 0.0512초
▶ 배열 : 0.0021초
▶ (환경에 따라 다르지만 통상 20~50배 차이)
```

개념

빠른 이유는 두 가지다. ① **같은 자료형만 담으니 메모리에 한 덩어리로 붙어 있어 값을 하나씩 찾아다닐 필요가 없다.** ② **반복이 파이썬이 아니라 C 수준에서 돌아간다.** 파이썬은 "이 배열에 2를 곱해"라고 한 번만 지시하고, 실제 100만 번의 곱셈은 훨씬 빠른 언어가 처리한다. 이것이 **벡터 연산**이고, 다음 시간의 주제다.

그래서 넘파이를 쓸 때는 사고방식 자체를 바꿔야 한다. "**값 하나하나를 어떻게 처리할까**"가 아니라 "**배열 전체에 무엇을 적용할까**"로 생각한다. 아래처럼 배열을 만들어 놓고 `for`로 돌리면, 넘파이를 쓰면서도 리스트만큼 느려진다 — 초보자가 가장 자주 하는 실수다.

같은 결과, 다른 사고방식

```
arr = np.array([70, 72, 71, 95, 73])
```

x 배열을 쓰면서 리스트처럼 — 느리고 길다

```
result = []
for x in arr:
    result.append(x * 2)
print(np.array(result))
```

✓ 배열답게 — 빠르고 짧다

```
print(arr * 2)
```

```
▶ [140 144 142 190 146]
▶ [140 144 142 190 146]
```

STEP 3

배열을 만드는 네 가지 함수

값 · 간격 · 개수 · 초기화

배열을 만드는 방법은 하나가 아니다. **내가 무엇을 알고 있느냐**에 따라 함수가 갈린다. 값을 이미 알면 `np.array`, 간격을 정하고 싶으면 `np.arange`, 개수로 나누고 싶으면 `np.linspace`, 일단 빈 그릇만 필요하면 `np.zeros`·`np.full`이다.

값을 안다
`np.array`

▶
간격이 중요
`np.arange`

▶
개수가 중요
`np.linspace`

▶
채워만 두기
`zeros` · `full`

3-1. np.arange — 간격을 정한다

함수 `np.arange()` 일정 간격의 연속된 숫자 생성

정의 시작부터 끝 직전까지 지정한 간격으로 값을 만든다. 끝값은 제외된다.

형태 `np.arange(끝)` / `np.arange(시작, 끝)` / `np.arange(시작, 끝, 간격)`

```
print(np.arange(5))
print(np.arange(0, 10, 2))
print(np.arange(0, 12, 2))
```

```
arr = np.arange(10)
print(arr.ndim, arr.dtype)
```

```
▶ [0 1 2 3 4]
▶ [0 2 4 6 8]
▶ [ 0  2  4  6  8 10]
▶ 1 int64
```

왜 쓰나 측정 시점처럼 **간격이 의미를 갖는** 데이터를 만들 때 쓴다. 1초 간격 샘플링, 5분 간격 집계 축이 전부 이 함수로 만들어진다.

응용 간격에 소수를 넣으면 부동소수점 오차 때문에 마지막 값이 미묘하게 어긋나거나 개수가 하나 더/덜 나올 수 있다.

정확히 나뉘어야 하는 구간은 `linspace`를 쓴다. 아래는 그 오차가 실제로 보이는 예다.

```
# 0.1 간격으로 0~1 을 만들면?
a = np.arange(0, 1, 0.1)
print(a.size)
print(a[-1])

# 0.3 간격은 더 위험하다
print(np.arange(0, 1, 0.3))
```

```
▶ 10
▶ 0.9000000000000001
▶ [0.  0.3 0.6 0.9]
```

자주 하는 실수

가장 흔한 실수 — `np.arange(5)`를 "0부터 5까지 여섯 개"로 착각하는 것. 실제로는 **0부터 4까지 다섯 개**. `range()`와 똑같은 규칙이고, 곧 배열 슬라이싱도 똑같다.

개념

"시작 포함, 끝 제외"는 `np.arange`만의 규칙이 아니라 파이썬 전체의 약속이다. `range(5)`, `lst[1:4]`, `arr[1:4]` 전부 같다. 이 약속 하나만 몸에 익히면 오늘 배울 슬라이싱에서 헛갈릴 일이 없다.

3-2. `np.linspace` — 개수를 정한다

함수 `np.linspace()` 시작과 끝 사이를 정해진 개수로 균등 분할

정의 시작값과 끝값 사이를 지정한 **개수**의 점으로 똑같이 나눈다. **끝값이 포함된다**.

형태 `np.linspace(시작, 끝, 개수)`

```
print(np.linspace(0, 1, 5))
print(np.linspace(0, 10, 5))
print(np.linspace(0, 30, 13))
```

```
▶ [0.  0.25 0.5  0.75 1.  ]
▶ [ 0.  2.5  5.  7.5 10. ]
▶ [ 0.  2.5  5.  7.5 10. 12.5 15.  17.5 20.  22.5 25.  27.5 30. ]
```

왜 쓰나 그래프 축을 만들 때는 보통 "몇 개로 나눌지"를 정하지 간격을 정하지 않는다. 그래서 시각화 코드에서는 `linspace`가 압도적으로 많이 쓰인다.

응용 `linspace`의 간격은 $(\text{끝} - \text{시작}) / (\text{개수} - 1)$ 이다. **개수가 아니라 개수 빼기 1로 나눈다** — 끝값이 포함되기 때문이다. 이 공식을 알면 결과를 미리 계산해 검산할 수 있다.

```
# 0~10 을 5개로 나누면 간격은?
# (10 - 0) / (5 - 1) = 2.5
a = np.linspace(0, 10, 5)
print(a)
print("간격:", a[1] - a[0])

# 간격도 함께 돌려받기
a, step = np.linspace(0, 10, 5, retstep=True)
print("retstep:", step)

▶ [ 0.  2.5  5.  7.5 10. ]
▶ 간격: 2.5
▶ retstep: 2.5
```

주의

세 번째 인자는 **간격이 아니라 점의 개수**다. `np.linspace(0, 10, 2)`는 "2 간격"이 아니라 `[0. 10.]` 두 개다. `arange`와 정반대라 가장 자주 헷갈린다.

3-3. arange와 linspace — 무엇을 정하는가

두 함수는 하는 일이 비슷해 보이지만 **내가 정하는 것과 자동으로 계산되는 것이 정확히 반대**다. 아래 표는 오늘 내용 중 시험에 나오기 가장 좋은 부분이다.

비교 항목		
내가 정하는 것	간격 (step)	개수 (num)
자동 계산되는 것	개수 — 세어 봐야 얇	간격 — 자동 분할
끝값	제외	포함
자료형	정수 인자면 int64	항상 float64
소수 간격	부동소수점 오차 위험	오차 없이 정확히 분할
주 용도	측정 시점 · 인덱스 축	그래프 축 · 균등 구간
예	<code>np.arange(0, 30, 6)</code> → [0 6 12 18 24]	<code>np.linspace(0, 30, 13)</code> → 13개

설비 적용

설비 데이터에서 **시간축**을 만들 때 둘의 선택이 같린다. "1초 간격으로 60초간 측정"은 간격이 정해진 것이므로 `np.arange(0, 60, 1)`, "0초부터 60초까지를 균등한 13개 구간으로 나눠 리포트"는 개수가 정해진 것이므로 `np.linspace(0, 60, 13)`이다. 강사가 실습 2·3에서 이 두 가지를 나란히 시킨 이유가 바로 이 감각을 잡게 하려는 것이다.

3-4. zeros · ones · full — 빈 그릇 먼저 만들기

함수 `np.zeros()` 0으로 채운 배열 생성

정의 지정한 개수만큼 0으로 채운 배열을 만든다. 기본 자료형은 실수(float64).

형태 `np.zeros(개수)` / `np.zeros(개수, 자료형)` / `np.zeros((행, 열))`

```
print(np.zeros(5))
print(np.zeros(3, int))
print(np.zeros((2, 3)))
```

```
▶ [0. 0. 0. 0. 0.]
▶ [0 0 0]
▶ [[0. 0. 0.]
   [0. 0. 0.]]
```

왜 쓰나 결과를 담을 빈 그릇을 미리 준비할 때 쓴다. 반복문을 돌며 값을 채워 넣을 자리를 먼저 만들어 두는 패턴이다.

응용 출력의 0.은 실수 배열이라는 표시다. 정수로 만들고 싶으면 두 번째 인자에 `int`를 넘긴다. 2차원은 **형태를 괄호로** 한 번 더 묶어 전달한다 — `np.zeros(2, 3)`이 아니라 `np.zeros((2, 3))`이다.

```
# 설비 5대 × 시점 4개 결과판 미리 만들기
result = np.zeros((5, 4))
print(result.shape)
```

```
# 자료형까지 지정
count = np.zeros((5, 4), int)
print(count.dtype)
```

```
▶ (5, 4)
▶ int64
```

자주 하는 실수

2차원에서 괄호를 한 겹 빼먹는 실수가 잦다. `np.zeros(2, 3)`은 3을 자료형으로 해석해 오류를 낸다.

함수 `np.ones()` · `np.full()` 1 또는 지정한 값으로 채우기

정의 `np.ones`는 1로, `np.full`은 내가 지정한 값으로 배열을 채운다.

형태 `np.ones(개수)` / `np.full(개수, 값)`

```
print(np.ones(4))
print(np.full(4, 7))
print(np.full(4, 7.0))
print(np.full(10, 10))
```

```
▶ [1.  1.  1.  1.]
▶ [7  7  7  7]
▶ [7.  7.  7.  7.]
▶ [10 10 10 10 10 10 10 10 10 10]
```

왜 쓰나 `np.ones`는 비율·가중치의 출발점으로, `np.full`은 **기준값을 설정할 때** 쓴다. 설비 임계치 배열을 만들 때가 대표적이다.

응용 `np.full(4, 7)`과 `np.full(4, 7.0)`의 출력이 다르다는 점에 주목하자. 채울 값이 정수면 `int` 배열, 실수면 `float` 배열이 된다. **채울 값이 자료형을 결정한다**. 실무에서는 기준선 배열을 만들어 실측 배열과 한 번에 비교하는 데 쓴다.

```
temp = np.array([68.0, 71.5, 82.3, 69.1])
limit = np.full(4, 75.0)  # 임계치 75도
```

```
print("실측:", temp)
print("기준:", limit)
print("초과:", temp - limit)
```

```
▶ 실측: [68.   71.5  82.3  69.1]
▶ 기준: [75.  75.  75.  75.]
▶ 초과: [-7.   -3.5   7.3  -5.9]
```

개념

`temp - limit`처럼 배열끼리 빼는 것이 **벡터 연산**이다. 다음 시간(10_02 PART 02)의 주제다.

3-5. 배열 생성 함수 정리 — 스스로 질문하기

교안 p22의 표현이 좋다 — "**함수를 고르는 가장 쉬운 방법은 스스로 질문을 던지는 것**"이다. 완벽히 외우지 못해도 된다. 아래 네 질문만 순서대로 던지면 답이 나온다.

스스로 던지는 질문	답이 예라면	설비 데이터 예
값을 이미 알고 있는가?	<code>np.array(리스트)</code>	센서 측정값 · 실측 기록
간격이 중요한가?	<code>np.arange(시작, 끝, 간격)</code>	1초 간격 측정 시점

스스로 던지는 질문	답이 예라면	설비 데이터 예
개수로 똑같이 나눌 것인가?	<code>np.linspace(시작, 끝, 개수)</code>	그래프 X축 · 균등 구간
같은 값으로 채워만 둘 것인가?	<code>np.zeros(n) / np.full(n, 값)</code>	결과 값을 빈 그릇 · 기준선

3-6. 수업 실습 되짚기 — `arange`와 `linspace`를 나란히

강사가 실습 2·3에서 두 함수를 연달아 시킨 이유가 있다. 같은 구간을 서로 다른 기준으로 나눠 보면 둘의 차이가 손에 잡힌다. 수업에서 실제로 쓴 숫자로 다시 해 보자.

실습 2 — 같은 구간, 다른 기준

```
import numpy as np
```

```
# 같은 구간 0~30 을 두 방식으로
```

```
a = np.arange(0, 30, 6)      # 간격 6
```

```
b = np.linspace(0, 30, 13)  # 점 13개
```

```
print(f"arange(0,30,6)   : {a}   (개수 {a.size})")
```

```
print(f"linspace(0,30,13): {b}   (개수 {b.size})")
```

```
print(f"arange  간격: {a[1] - a[0]}")
```

```
print(f"linspace 간격: {b[1] - b[0]}")
```

```
▶ arange(0,30,6)   : [ 0  6 12 18 24]  (개수 5)
```

```
▶ linspace(0,30,13): [ 0.   2.5  5.   7.5 10.  12.5 15.  17.5 20.  22.5 25.  27.5
30. ]  (개수 13)
```

```
▶ arange  간격: 6
```

```
▶ linspace 간격: 2.5
```

`arange`는 끝값 30이 빠졌고(24까지), `linspace`는 30이 들어 있다. 개수도 5개와 13개로 다르다. 같은 "0부터 30까지"인데 결과가 이렇게 다르다는 것을 눈으로 확인해 두면 헛갈릴 일이 없다.

실습 3 — 간격을 바꾸면 개수가 어떻게 변하나

```
# 실습 3 — 측정 시간축, 간격을 바꿔가며 개수 관찰  
start, end = 0, 10
```

```
for interval in [2, 1, 0.5]:  
    axis = np.arange(start, end, interval)  
    print(f"간격 {interval}초 → 개수 {axis.size:2d} · {axis}")
```

- ▶ 간격 2초 → 개수 5 · [0 2 4 6 8]
- ▶ 간격 1초 → 개수 10 · [0 1 2 3 4 5 6 7 8 9]
- ▶ 간격 0.5초 → 개수 20 · [0. 0.5 1. 1.5 ... 9.5]

설비 적용

간격이 절반이 되면 개수는 두 배가 된다. **샘플링 주기를 1초에서 0.5초로 바꾸면 하루 데이터가 두 배로 늘어난다**는 뜻이다. 설비 데이터 수집 설계에서 주기를 정하는 일은 곧 **저장 용량·전송 대역·분석 시간**을 정하는 일이다.

`arange`로 개수를 미리 계산해 보는 습관이 여기서 쓰인다.

주의

실습 3에서 `interval`을 0.5로 바꾸면 배열의 `dtype`이 `int64`에서 `float64`로 **자동으로 바뀐다**. 간격이 실수면 결과도 실수가 되기 때문이다. 정수 인덱스로 쓰려던 배열이 갑자기 실수가 되어 오류가 나는 경우가 여기서 생긴다.

STEP 4

배열의 구조를 읽는 네 속성

데이터를 만나면 가장 먼저 확인하는 것

교안은 이 단계를 "운전 전 거울·계기판 확인"에 비유한다. 새 데이터를 받으면 계산부터 하는 게 아니라, 이게 몇 차원인지·몇 행 몇 열인지·값이 몇 개인지·자료형이 무엇인지를 먼저 확인한다. 실무 분석가가 데이터를 처음 만나면 던지는 첫 질문이 "shape이 뭐지?" 인 이유다.



4-1. 1차원과 2차원 — 차원이란 무엇인가

차원을 쉽게 풀면 **값을 늘어놓는 방향의 개수**다. 1차원은 한 줄로 선 사람들처럼 앞뒤 순서만 있고, 2차원은 극장 좌석처럼 "몇 번째 줄, 몇 번째 자리"라는 두 정보로 위치가 정해진다.

구분	1차원	2차원
형태	값이 한 줄로 늘어선 형태	행과 열을 가진 표 모양
설비 데이터 의미	센서 하나가 시간에 따라 측정한 값	행=시점, 열=센서
만드는 법	<code>np.array([1, 2, 3])</code>	<code>np.array([[1, 2], [3, 4]])</code>
<code>.ndim</code>	1	2
위치 지정	<code>a[2]</code> 번호 하나	<code>data[1, 0]</code> 행·열 두 개

1차원과 2차원

```
import numpy as np
```

```
# 1차원 — 센서 하나의 시간별 측정값
```

```
one_d = np.array([70, 72, 71, 95, 73])
```

```
# 2차원 — 행=시점, 열=센서(온도, 진동)
```

```
two_d = np.array([
    [70, 2.1],
    [72, 2.3],
    [71, 2.0],
    [95, 4.8]
])
```

```
print(one_d.ndim, two_d.ndim)
print(two_d)
```

```
▶ 1 2
▶ [[70.  2.1]
   [72.  2.3]
   [71.  2. ]
   [95.  4.8]]
```

설비 적용

실무에서 다루는 설비 데이터는 **거의 항상 2차원**이다. 행이 측정 시점, 열이 센서 종류인 표 모양이다. 1차원은 그 표에서 한 줄(한 시점의 모든 센서값)이나 한 칸 열(한 센서의 모든 시점값)을 떼어낸 것이라고 생각하면 된다. STEP 7의 `data[:, 0]`이 정확히 이 "한 센서의 전체 시점값"을 뽑는 표현이다.

4-2. 네 가지 속성 — 전부 괄호가 없다

속성 `.ndim` 배열이 몇 차원인지

정의 number of dimensions의 줄임말. 값을 늘어놓은 방향의 개수를 정수로 돌려준다.

형태 배열.`ndim`

```
a = np.array([1, 2, 3])
b = np.array([[1, 2], [3, 4]])
```

```
print(a.ndim)
print(b.ndim)
```

```
▶ 1
▶ 2
```

왜 쓰나 이후 작업 방향이 여기서 갈린다. 1차원이면 `a[2]`, 2차원이면 `data[1, 0]`처럼 **위치 지정 방식 자체가 달라진다**. 그래서 가장 먼저 확인한다.

응용 3차원 이상도 존재한다. 이미지 데이터가 대표적이다 — 가로×세로×색상(RGB) 세 방향이라 `ndim`이 3이다. 컬러 이미지 여러 장을 한꺼번에 다루면 4차원이 된다. 딥러닝 입력 데이터의 `shape`이 (배치, 높이, 너비, 채널) 네 칸인 이유가 이것이다.

```
# 컬러 이미지 한 장: 세로 2 × 가로 2 × RGB 3
img = np.zeros((2, 2, 3), int)
print(img.ndim, img.shape)

# 이미지 10장 묶음 (배치)
batch = np.zeros((10, 2, 2, 3), int)
print(batch.ndim, batch.shape)
```

```
▶ 3 (2, 2, 3)
▶ 4 (10, 2, 2, 3)
```

주의

`ndim` 뒤에 **괄호가 없다는 것이 핵심**이다. `a.ndim()`이라고 쓰면 `TypeError`가 난다.

속성 `.shape` 행·열 크기 — 앞이 행, 뒤가 열

정의 배열의 형태를 튜플로 돌려준다. 2차원이면 (행, 열) 순서.

형태 배열 `.shape`

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.shape)

a = np.array([70, 72, 71, 95, 73])
print(a.shape)
```

```
▶ (2, 3)
▶ (5,)
```

왜 쓰나 분석가가 데이터를 처음 만나면 가장 먼저 던지는 질문이 **"shape이 뭐지?"** 다. (1000, 20)이면 "시점 1000개, 센서 20개"라고 데이터 규모가 한눈에 잡힌다.

응용 1차원 배열의 `shape`이 (5,)처럼 쉼표로 끝나는 것은 오타가 아니다. **값이 하나뿐인 튜플**이라는 파이썬 표기다(9일차 튜플 참고). 그리고 `shape`은 튜플이므로 인덱싱해서 행 수·열 수만 따로 꺼낼 수 있다 — 실무에서 자주 쓰는 패턴이다.

```
data = np.array([[70, 2.1], [72, 2.3], [71, 2.0]])
```

```
rows, cols = data.shape    # 튜플 언패킹  
print(f"시점 {rows}개, 센서 {cols}개")
```

```
# 인덱싱으로 따로 꺼내도 된다  
print(data.shape[0], data.shape[1])
```

```
▶ 시점 3개, 센서 2개  
▶ 3 2
```

개념

shape의 (행, 열) 순서는 **2차원 인덱싱** data[행, 열]과 정확히 같은 순서다. 두 개념이 같은 약속을 쓰니 함께 기억하면 절대 안 헷갈린다.

속성 `.size` 값의 전체 개수 — 행 × 열

정의 배열에 담긴 값의 총 개수. 2차원이면 행 × 열, 즉 표 안의 칸 총 개수.

형태 배열.size

```
b = np.array([[1, 2, 3], [4, 5, 6]])  
print(b.shape)  
print(b.size)
```

```
# shape 의 숫자들을 곱하면 size  
print(2 * 3)
```

```
▶ (2, 3)  
▶ 6  
▶ 6
```

왜 쓰나 shape은 "어떤 모양인가", size는 "값이 모두 몇 개인가"를 알려준다. 데이터 양을 가늠하고 메모리를 예측할 때 쓴다.

응용 shape과 size의 관계가 핵심이다 — **shape의 숫자들을 모두 곱하면 size**다. (500, 12)면 size는 6000.

그리고 **모양은 바꿀 수 있어도 size는 변하지 않는다**. 이 보존 법칙이 STEP 5의 reshape을 이해하는 열쇠다.

```

a = np.arange(12)
print(a.shape, a.size)

b = a.reshape(3, 4)
print(b.shape, b.size)    # 모양은 변했지만 size 동일

c = a.reshape(2, 6)
print(c.shape, c.size)

```

```

▶ (12,) 12
▶ (3, 4) 12
▶ (2, 6) 12

```

주의

`len(배열)`과 헷갈리지 말 것. 2차원 배열에서 `len()`은 **행의 개수**만 돌려주고, `size`는 전체 칸 수를 돌려준다.

속성 `.dtype` 값의 자료형 — `int64` / `float64`

정의 배열이 담고 있는 값의 종류. 정수는 `int`, 실수는 `float`, 뒤의 64는 저장 공간 크기(비트).

형태 배열.`dtype`

```

a = np.array([1, 2, 3])
b = np.array([1.5, 2.7, 3.1])
c = np.array(["70", "72"])

print(a.dtype)
print(b.dtype)
print(c.dtype)

```

```

▶ int64
▶ float64
▶ <U2

```

왜 쓰나 데이터를 불러온 직후 **dtype 확인이 계산 오류를 막아 준다**. CSV에서 읽어 온 값은 숫자처럼 보여도 글자로 저장된 경우가 많고, 그대로 계산하면 엉뚱한 결과가 나온다.

응용 세 번째 출력 `<U2`가 그 위험 신호다 — "유니코드 문자열 최대 2글자"라는 뜻이다. `dtype`이 `int`·`float`이 아니면 **숫자가 아니다**. 이 상태로 더하면 문자열이 이어 붙어 버린다. 13일차에 배운 파일 입출력과 만나는 지점이 정확히 여기서다.

```
# CSV 에서 읽으면 이런 상태일 수 있다
raw = np.array(["70", "72", "71"])
print(raw.dtype)
print(raw + raw)          # 숫자가 아니라 글자가 이어 붙는다

# 숫자로 바뀌야 계산이 된다
num = raw.astype(float)
print(num.dtype)
print(num + num)

▶ <U2
▶ ['7070' '7272' '7171']
▶ float64
▶ [140. 144. 142.]
```

자주 하는 실수

dtype은 모양과 무관한 독립 정보다. ndim·shape·size는 서로 연결되어 있지만 dtype만 별개다. 그래서 네 가지를 다 확인해야 한다.

4-3. 배열 속성 종합 정리

속성	알려주는 것	괄호	서로의 관계
.ndim	차원 개수	없음	shape의 숫자 개수와 같다
.shape	행·열 크기 (튜플)	없음	곱하면 size
.size	값 총 개수	없음	reshape해도 변하지 않는다
.dtype	자료형	없음	모양과 무관한 독립 정보

4-4. astype — 자료형 바꾸기

메서드 .astype() 자료형 변환 — 새 배열을 만든다

정의 배열의 자료형을 바꾼 새 배열을 돌려준다. 실수를 정수로 바꾸면 소수점 아래를 버린다.

형태 배열.astype(자료형)

```
a = np.array([1.7, 2.3, 3.9])
print(a.astype(int))

b = np.array([99.9, 12.5])
print(b.astype(int))

▶ [1 2 3]
▶ [99 12]
```

왜 쓰나 파일에서 읽어 온 문자열을 숫자로 바꾸거나, 소수 계산 결과를 개수·순번 같은 정수로 정리할 때 쓴다.

응용 반올림이 아니라 버림이라는 점이 결정적이다. 1.7은 2가 아니라 1이 되고, 99.9는 100이 아니라 99가 된다.

반올림이 필요하면 `np.round()`를 거친 뒤 `astype`을 한다. 정밀한 측정값을 함부로 정수로 바꾸면 데이터가 조용히 망가진다.

```
a = np.array([1.7, 2.3, 99.9])

print(a.astype(int))          # 버림
print(np.round(a).astype(int)) # 반올림

# 원본은 그대로 — 결과를 담아야 쓸 수 있다
print(a)
z = a.astype(int)
print(z)
```

```
▶ [ 1  2 99]
▶ [  2  2 100]
▶ [ 1.7  2.3 99.9]
▶ [ 1  2 99]
```

자주 하는 실수

`a.astype(int)`만 실행하고 **아무 데도 담지 않으면 결과가 사라진다**. 원본 `a`는 그대로다. `z = a.astype(int)`처럼 새 이름에 담아야 한다. `reshape`·`flatten`도 전부 마찬가지다.

개념

오늘 나온 메서드 `astype`·`reshape`·`flatten`은 **셋 다 원본을 바꾸지 않고 새 배열을 만든다**. 9일차 리스트의 `sort()`가 원본을 직접 바꿨던 것과 정반대다. 넘파이는 "원본 보존 + 새 배열 반환"이 기본이라고 통째로 외워 두면 편하다.

STEP 5

모양 바꾸기

값은 그대로, 배치만 바꾸기

파일에서 데이터를 불러오면 처음부터 원하는 모양인 경우가 드물다. 한 줄로 쭉 이어진 값을 「행=시점, 열=센서」 표로 정리해야 분석이 시작된다. 그 일을 하는 것이 `reshape`이고, 반대로 표를 한 줄로 펴는 것이 `flatten`이다.

5-1. reshape — 한 줄을 표로

메서드 `.reshape()` 배열의 모양 변경

정의 값은 그대로 두고 행·열 배치만 바꾼 새 배열을 돌려준다. **바꾸기 전후 값 개수(size)가 같아야 한다.**

형태 배열.`.reshape(행, 열)`

```
a = np.arange(12)
print(a)

print(a.reshape(3, 4))
print(a.reshape(2, 6))

▶ [ 0  1  2  3  4  5  6  7  8  9 10 11]
▶ [[ 0  1  2  3]
   [ 4  5  6  7]
   [ 8  9 10 11]]
▶ [[ 0  1  2  3  4  5]
   [ 6  7  8  9 10 11]]
```

왜 쓰나 센서 로그는 보통 한 줄로 쏟아진다. 그것을 「행=시점, 열=센서」 표로 세우는 순간부터 행별·열별 통계가 가능해진다.

응용 행이나 열 자리에 `-1`을 넣으면 넘파이가 나머지를 자동 계산한다. 데이터가 많아 개수를 세기 어려울 때 "열은 4개로 고정, 행은 알아서" 라고 말기는 것이다. 단, `-1`은 행과 열 중 한 곳에만 쓸 수 있다.

```

a = np.arange(12)

print(a.reshape(-1, 4))    # 열 4개 고정, 행은 자동(3)
print(a.reshape(2, -1))   # 행 2개 고정, 열은 자동(6)

# 센서 3개 로그가 한 줄로 들어왔을 때
log = np.arange(9)
print(log.reshape(-1, 3)) # 시점 3 × 센서 3

```

```

▶ [[ 0  1  2  3]
▶ [ 4  5  6  7]
▶ [ 8  9 10 11]]
▶ [[ 0  1  2  3  4  5]
▶ [ 6  7  8  9 10 11]]
▶ [[0 1 2]
▶ [3 4 5]
▶ [6 7 8]]

```

자주 하는 실수

12개를 3행 5열(15칸)로는 못 바꾼다. `ValueError: cannot reshape array of size 12 into shape (3,5)` 오류는 거의 전부 이 문제다. **바꾸기 전** `.size`를 먼저 확인하는 습관을 들이자.

개념

size 보존 법칙. 모양은 바꿀 수 있어도 담긴 값의 총 개수는 절대 변하지 않는다. 블록 12개를 3×4 로 쌓든 2×6 으로 쌓든 블록은 여전히 12개다. `reshape`이 되는지 안 되는지는 "**행 × 열 = size 인가**" 한 줄로 판정된다.

5-2. flatten — 표를 한 줄로

메서드 `.flatten()` 2차원을 1차원으로 펴기

정의 표 모양 배열을 한 줄로 편다. 위 행부터, 그 안에서 왼쪽에서 오른쪽으로 — 책 읽는 방향이다.

형태 배열 `.flatten()`

```

b = np.array([[1, 2, 3], [4, 5, 6]])
print(b)
print(b.flatten())

```

```

▶ [[1 2 3]
▶ [4 5 6]]
▶ [1 2 3 4 5 6]

```

왜 쓰나 여러 센서값이 표로 있는데 **센서 구분 없이 전체 평균**을 구하고 싶을 때 쓴다. 한 줄로 편 뒤 계산하면 된다.

응용 퍼지는 순서를 알면 결과를 미리 예상할 수 있고, 퍼진 값이 원래 어느 위치에서 왔는지 역추적할 수도 있다.

flatten한 결과의 i번째 값은 원래 $(i // \text{열수}, i \% \text{열수})$ 위치에 있던 값이다. 이 공식은 이미지 처리·딥러닝 입력 변환에서 계속 쓰인다.

```
b = np.array([[10, 20, 30], [40, 50, 60]])
flat = b.flatten()
print(flat)

# 5번째 값은 원래 어디에 있었나?
i, cols = 4, b.shape[1]
print(f"flat[{i}] = {flat[i]} ← b[{i // cols}, {i % cols}] = {b[i // cols, i % cols]}")

# 다시 표로 되돌리기
print(flat.reshape(b.shape))
```

```
▶ [10 20 30 40 50 60]
▶ flat[4] = 50 ← b[1, 1] = 50
▶ [[10 20 30]
▶  [40 50 60]]
```

개념

reshape이 한 줄을 표로 만든다면 flatten은 반대로 표를 한 줄로 편다. 둘 다 값 자체는 바꾸지 않고 배치만 바꾸며, 값의 총 개수는 변하지 않는다.

5-3. 형태 변환 정리

메서드	하는 일	결과	원본
<code>.reshape(행, 열)</code>	모양 바꾸기	원하는 행·열의 2차원	안 바뀜
<code>.reshape(-1, 열)</code>	행을 자동 계산	열만 고정, 행은 넘파이가 계산	안 바뀜
<code>.flatten()</code>	한 줄로 퍼기	1차원 배열	안 바뀜

공통점이 중요하다 — 셋 다 값 자체는 바꾸지 않고 배치만 바꾸며, 값의 총 개수는 변하지 않고, 결과를 새 이름에 담아야 쓸 수 있다. 한 줄로 들어온 데이터를 표로 정리할 땐 reshape, 표를 한 줄로 퍼서 전체를 다룰 땐 flatten이다.

설비 적용

설비 로그 파일은 보통 시각, 온도, 진동, 압력 형태로 한 줄씩 쌓인다. 이걸 읽어서 숫자만 뽑으면 `[70, 2.1, 1.3, 71, 2.3, 1.4, ...]` 처럼 한 줄로 이어진 배열이 된다. 여기에 `reshape(-1, 3)`을 걸면 행=시점, 열=센서 3개인 표가 즉시 만들어진다. 다음 시간에 배울 `data[:, 0]`으로 온도만 뽑아 통계를 내는 흐름이 여기서 시작된다.

만들기 → 확인 → 정리, 한 흐름으로

배열 다루기 전체 흐름 — 만들기 → 확인 → 정리

```
import numpy as np
```

① 만들기

```
log = np.array([70, 2.1, 1.3, 72, 2.3, 1.4, 71, 2.0, 1.2])
```

② 구조 확인

```
print("ndim :", log.ndim)
print("shape:", log.shape)
print("size :", log.size)
print("dtype:", log.dtype)
```

③ 모양 정리 — 시점 3 × 센서 3

```
table = log.reshape(-1, 3)
print(table)
print("정리 후 shape:", table.shape)
```

```
▶ ndim : 1
▶ shape: (9,)
▶ size : 9
▶ dtype: float64
▶ [[70.  2.1  1.3]
   [72.  2.3  1.4]
   [71.  2.   1.2]]
▶ 정리 후 shape: (3, 3)
```

5-4. reshape 실전 — 파일에서 읽은 한 줄을 표로

13일차에 배운 파일 입출력과 오늘의 `reshape`이 만나는 지점이다. CSV에서 숫자만 뽑으면 한 줄로 이어진 배열이 되는데, 여기에 `reshape(-1, 센서수)`를 걸면 곧바로 분석용 표가 된다.

한 줄 로그 → 시점 × 센서 표

```
import numpy as np
```

```
# 파일에서 읽었다고 가정한 한 줄 데이터 (온도, 진동, 압력) × 4시점
```

```
raw = np.array([
    70.5, 2.1, 1.30,
    69.8, 2.3, 1.35,
    73.7, 2.0, 1.28,
    95.2, 4.8, 1.90
])
```

```
SENSORS = 3
```

```
print(f"읽어온 값 {raw.size}개 · shape {raw.shape}")
```

```
table = raw.reshape(-1, SENSORS)
```

```
print(table)
```

```
print(f"정리 후: 시점 {table.shape[0]} · 센서 {table.shape[1]}")
```

```
▶ 읽어온 값 12개 · shape (12,)
```

```
▶ [[70.5  2.1  1.3 ]
```

```
▶ [69.8  2.3  1.35]
```

```
▶ [73.7  2.   1.28]
```

```
▶ [95.2  4.8  1.9 ]]
```

```
▶ 정리 후: 시점 4 · 센서 3
```

설비 적용

센서 개수를 `SENSORS = 3`처럼 상수로 빼 두는 것이 핵심이다. 센서가 추가되면 그 숫자 하나만 고치면 되고, `reshape(-1, SENSORS)`의 `-1` 덕분에 시점 수는 자동으로 계산된다. 데이터가 몇 줄이든 코드를 고칠 필요가 없다 — 실무 코드가 갖춰야 할 최소한의 유연성이다.

STEP 6

값 꺼내기 — 인덱싱

번호 하나로 콧 집기 · 1차원과 2차원

여기서부터 교안 10_02 「데이터 추출과 통계 분석」이다. 배열을 만들고 모양을 다뤘다면, 이제 그 안에서 **원하는 값만 콧 집어 꺼내는** 단계다. 강사의 비유를 빌리면 "3분단 2번째 줄 세 번째 학생 일어나서 읽어" 하는 것과 같다.

6-1. 1차원 인덱싱 — 번호는 0부터, 음수는 뒤에서부터

문법 배열[번호] 특정 위치의 값 하나 꺼내기

정의 대괄호에 번호를 넣어 값 하나를 꺼낸다. 번호(인덱스)는 **0부터 시작**하고, **음수는 뒤에서부터** 센다.

형태 배열[인덱스]

```
import numpy as np

temp = np.array([70, 72, 71, 95, 73])
print(temp)

print(temp[0])    # 첫 번째
print(temp[3])    # 네 번째
print(temp[-1])   # 맨 마지막
print(temp[-2])   # 뒤에서 두 번째
```

```
▶ [70 72 71 95 73]
▶ 70
▶ 95
▶ 73
▶ 95
```

왜 쓰나 -1은 데이터 개수와 상관없이 항상 맨 마지막 값이다. 센서 로그에서 "가장 최근 측정값"을 꺼낼 때 개수를 세지 않아도 되니 실무에서 압도적으로 자주 쓴다.

응용 설비 모니터링에서 가장 흔한 패턴이 "**최신값과 직전값 비교**"다. [-1]과 [-2]만 있으면 급변 감지 로직 한 줄이 나온다. 배열 길이가 계속 늘어나도 코드를 고칠 필요가 없다는 것이 핵심이다.

```
temp = np.array([70, 72, 71, 95, 73])

latest = temp[-1]
before = temp[-2]
print(f"최신 {latest}도, 직전 {before}도, 변화 {latest - before}도")

if abs(latest - before) > 10:
    print("급변 감지")
else:
    print("정상 범위")
```

▶ 최신 73도, 직전 95도, 변화 -22도
▶ 급변 감지

자주 하는 실수

인덱스가 범위를 넘으면 `IndexError: index 5 is out of bounds for axis 0 with size 5`가 뜬다. 값이 5개면 쓸 수 있는 번호는 **0~4**다.

비유

강사의 설명이 좋았다 — 음수 인덱스는 "**후진**"이다. 0번에서 한 칸 뒤로 가면 앞에 데이터가 없으니 빙 돌아 맨 끝으로 간다. -1이 마지막, -2가 그 앞이다. 앞으로 세면 0부터, 뒤로 세면 -1부터 — 0은 앞쪽에만 있으니 뒤에는 -0이 없다고 기억하면 된다.

6-2. 2차원 인덱싱 — 앞이 행, 뒤가 열

2차원 배열에서 값 하나를 꺼내는 방법은 **두 가지**다. 리스트처럼 대괄호를 두 번 쓰는 방법과, 넘파이의 쉼표 표기다.

결과는 같지만 강사가 분명히 말했다 — "**교안에는 아래 방식만 언급돼 있다. 배열에서는 아래 방식만 생각하시면 된다.**"

문법 배열[행, 열] 2차원에서 값 하나 꺼내기

정의 쉼표로 행과 열을 한 번에 지정한다. **앞이 행(row)**, **뒤가 열(column)**. `shape`의 순서와 동일하다.

형태 배열[행번호, 열번호]

```
data = np.array([
    [70, 2.1],
    [72, 2.3]
])

print(data)
print(data.dtype)

print(data[0][1])    # 리스트 방식 — 대괄호 두 번
print(data[0, 1])    # 넘파이 방식 — 쉼표 (권장)
```

```
▶ [[70.  2.1]
   [72.  2.3]]
▶ float64
▶ 2.1
▶ 2.1
```

왜 쓰나 쉼표 표기가 **수학의 행렬 표기와 같은 형태**라 통계·수학 배경을 가진 사람에게 훨씬 자연스럽다. 넘파이는 원래 그 사람들을 위해 만들어진 도구다. 실무 코드와 문서도 대부분 쉼표 표기를 쓴다.

응용 `data[0][1]`은 사실 **두 번 일한다** — 먼저 0행 전체를 꺼내 임시 배열을 만들고, 거기서 다시 1번을 꺼낸다.

`data[0, 1]`은 한 번에 위치를 계산한다. 값이 많아지면 성능 차이가 나고, 무엇보다 **슬라이싱과 섞어 쓸 때 [][]** 방식은 원하는 대로 동작하지 않는다.

```
data = np.array([[70, 2.1], [72, 2.3], [71, 2.0]])

# 모든 행의 0열을 뽑고 싶다
print(data[:, 0])    # 쉼표 방식 — 의도대로

# 대괄호 두 번으로는 이게 안 된다
print(data[][0])     # 전체를 꺼낸 뒤 0행 → 전혀 다른 결과
```

```
▶ [70. 72. 71.]
▶ [70.  2.1]
```

주의

`data[0][1]`과 `data[0, 1]`은 값 하나를 꺼낼 때만 결과가 같다. **슬라이싱이 끼면 완전히 달라진다**. 처음부터 쉼표 표기로 통일하는 것이 안전하다.

6-3. 행과 열 — 두 가지만 기억한다

기억할 것	내용	같은 규칙을 쓰는 곳
① 순서	항상 행(row)이 먼저, 열(column)이 나중	.shape도 (행, 열) 순서
② 시작 번호	번호는 0부터. 왼쪽 위 칸이 0행 0열	arange · 슬라이싱도 0부터

코드에서는 행을 row 또는 r, 열을 col 또는 c로 쓰는 관례가 있다. 강사가 짚었듯 column을 그대로 쓰면 row(3글자)와 길이가 안 맞아 보기 나쁘기 때문에 col로 줄여 맞추는 경우가 많다. **남의 코드에서 r, c가 보이면 행과 열이라고 바로 읽을 수 있어야 한다.**

교안 실습 2 데이터로 연습

```
data = np.array([
    [1151, 42.8], # 0행 — 설비 1호 (회전수, 토크)
    [1408, 46.3], # 1행 — 설비 2호
    [2861, 4.6], # 2행 — 설비 3호 ← 이상값
    [1410, 65.7] # 3행 — 설비 4호
]) # 0열 1열
```

```
print("2행 0열 (설비 3호 회전수):", data[2, 0])
print("3행 1열 (설비 4호 토크) :", data[3, 1])
print("맨 마지막 행의 마지막 값 :", data[-1, -1])
```

```
▶ 2행 0열 (설비 3호 회전수): 2861.0
▶ 3행 1열 (설비 4호 토크) : 65.7
▶ 맨 마지막 행의 마지막 값 : 65.7
```

설비 적용

위 데이터에서 설비 3호를 보자 — 회전수 2861로 다른 설비의 두 배인데 토크는 4.6으로 10분의 1이다. **회전은 빠르는데 힘이 안 걸린다**는 뜻이니, 공구가 부러졌거나 소재를 헛도는 상황을 의심할 수 있다. 인덱싱은 이런 값을 **한 칸씩 확인할 때** 쓰고, 다음 시간에 배울 조건 필터링은 이런 값을 **한 번에 찾아낼 때** 쓴다.

STEP 7

구간 꺼내기 — 슬라이싱

콜론이 전부를 의미한다

인덱싱이 값 **하나**를 꺼내는 것이라면, 슬라이싱은 **여기부터 저기까지 한 덩어리**를 잘라 오는 것이다. 강사의 비유대로 슬라이스 치즈처럼 얇게 잘라 내는 것이다. 리스트에서 이미 배운 개념이라 규칙은 그대로지만, 2차원에서 **행과 열을 각각 자를 수 있다**는 점이 새로 붙는다.

7-1. 1차원 슬라이싱 — 시작:끝, 끝은 제외

문법 배열[시작:끝] 구간 잘라내기

정의 시작 번호부터 끝 번호 **직전**까지 잘라 온다. 시작은 포함, 끝은 제외.

형태 배열[시작:끝]

```
temp = np.array([70, 72, 71, 95, 73, 68])
print(temp)
```

```
print(temp[1:4])    # 1·2·3번 (4번 제외)
print(temp[:3])     # 처음부터 2번까지
print(temp[3:])     # 3번부터 끝까지
print(temp[:])      # 전체
```

```
▶ [70 72 71 95 73 68]
▶ [72 71 95]
▶ [70 72 71]
▶ [95 73 68]
▶ [70 72 71 95 73 68]
```

왜 쓰나 "최근 10분 구간만", "앞쪽 100개만 샘플로" 같은 요구가 전부 슬라이싱 한 줄로 끝난다. 반복문으로 범위를 세면서 답을 필요가 없다.

응용 시작이나 끝을 **생략하면 처음부터 / 끝까지**가 된다. 음수와 조합하면 "마지막 N개"를 개수와 무관하게 뽑을 수 있다 — 실시간 모니터링에서 최근 구간만 보는 표준 패턴이다.

```
temp = np.array([70, 72, 71, 95, 73, 68])

print(temp[-3:])    # 마지막 3개
print(temp[:-3])    # 마지막 3개를 뺀 나머지
print(temp[-1:])    # 마지막 1개 (배열로 나옴)
print(temp[-1])     # 마지막 값 (스칼라로 나옴)
```

```
▶ [95 73 68]
▶ [70 72 71]
▶ [68]
▶ 68
```

자주 하는 실수

강사도 수업 중 헛갈려 정정했던 부분이다. `temp[1:4]`는 **네 개가 아니라 세 개**다. "1번부터 4번까지"가 아니라 "**1번부터 4번 직전까지**" 라고 읽는 습관을 들이자.

개념

`temp[-1:]`과 `temp[-1]`의 차이에 주목하자. **슬라이싱은 항상 배열을 돌려주고, 인덱싱은 값 하나를 돌려준다.** 대괄호 안에 콜론이 있으면 배열, 없으면 값이다. 이 차이 때문에 `temp[-1] + 5`는 되지만 `temp[-1:] + 5`는 배열 연산이 된다.

문법 배열[: :간격] 간격을 두고 건너뛰며 뽑기

정의 콜론 두 개 뒤의 숫자가 간격. : : 2면 두 칸씩 건너뛴다.

형태 배열[시작:끝:간격]

```
temp = np.array([70, 72, 71, 95, 73, 68])

print(temp[::2])    # 0, 2, 4번
print(temp[1::2])    # 1, 3, 5번
print(temp[:, :3])   # 0, 3번
print(temp[:, :-1])  # 거꾸로
```

```
▶ [70 71 73]
▶ [72 95 68]
▶ [70 95]
▶ [68 73 95 71 72 70]
```

왜 쓰나 강사가 이 부분에서 **간격의 중요성**을 길게 설명했다. 센서가 1초마다 값을 쏟아내는데 한 시간 단위 비교만 하면 되는 경우, 중간 값을 전부 계산하는 건 의미가 없다. **일정 간격으로 속아내는 것(다운샘플링)**이 실무의 첫 단계다.

응용 `::-1`은 간격을 -1로 준 것이라 **배열이 통째로 뒤집힌다**. 최신값부터 거꾸로 훑을 때 쓴다. 그리고 `arange`의 "간격"과 슬라이싱의 "간격"은 같은 개념이지만, `arange`는 **값을 만들 때**, 슬라이싱은 **이미 있는 값에서 뺄 때** 쓴다는 차이가 있다.

```
# 1초 간격 60초치 데이터를 10초 간격으로 다운샘플링
sec = np.arange(0, 60)      # 만들 때: arange
print("원본 개수:", sec.size)

down = sec[::-10]           # 뺄 때: 슬라이싱
print("10초 간격:", down)
print("줄어드는 개수:", down.size)
```

▶ 원본 개수: 60
▶ 10초 간격: [0 10 20 30 40 50]
▶ 줄어드는 개수: 6

개념

슬라이싱 결과를 수정하면 **원본도 함께 바뀔 수 있다**(뷰, view). 리스트 슬라이싱은 복사본을 만들지만 넘파이는 원본을 가리키는 창을 돌려준다. 원본을 지키려면 `.copy()`를 붙인다.

7-2. 슬라이싱의 두 가지 약속

규칙	내용	같은 규칙이 적용되는 곳
① 시작 포함, 끝 제외	<code>a[1:4]</code> 는 1·2·3번. 4번은 안 들어온다	<code>range()</code> · <code>np.arange()</code> · 리스트 슬라이싱
② 생략 가능, 콜론 둘은 간격	<code>a[:3]</code> 처음부터 · <code>a[3:]</code> 끝까지 · <code>a[::-2]</code> 간격	리스트·문자열 슬라이싱과 동일

7-3. 2차원 슬라이싱 — 콜론은 "전부"를 의미한다

여기가 오늘의 마지막이자 가장 중요한 부분이다. 2차원에서 **콜론(:)**은 **"그 방향의 모든 경우"**를 뜻한다. `data[:, 0]`을 읽으면 "모든 행에서, 0번 열만"이 된다.

문법 `배열[행번호]` 행 전체 꺼내기

정의 행 번호만 쓰면 그 행 전체가 1차원 배열로 나온다. 행은 배열의 기본 단위라 번호만으로 추출된다.

형태 `배열[행번호]`

```
data = np.array([
    [70, 2.1],
    [72, 2.3]
])

print(data[0])    # 0행 전체
print(data[1])    # 1행 전체
print(data[0].shape)
```

```
▶ [70.  2.1]
▶ [72.  2.3]
▶ (2,)
```

왜 쓰나 설비 데이터에서 한 행은 **그 시점의 모든 센서값**이다. "3번째 측정 시점에 온도·진동·압력이 각각 얼마였나"를 꺼내는 것이 행 추출이다.

응용 `data[0]`은 사실 `data[0, :]`의 줄임이다. 뒤의 `:`를 생략해도 되기 때문에 짧게 쓸 수 있는 것이다. 이 사실을 알면 행 슬라이싱도 자연스럽게 이해된다 — `data[0:2]`는 0·1행을 통째로 잘라 2차원으로 돌려준다.

```
data = np.array([[70, 2.1], [72, 2.3], [71, 2.0], [95, 4.8]])

print(data[0])      # 0행 (1차원)
print(data[0, :])   # 같은 뜻
print(data[0:2])    # 0·1행 (2차원 유지)
print(data[0:2].shape)
```

```
▶ [70.  2.1]
▶ [70.  2.1]
▶ [[70.  2.1]
▶  [72.  2.3]]
▶ (2, 2)
```

주의

`data[0]`은 1차원 `(2,)`, `data[0:2]`는 2차원 `(2, 2)`다. 번호 하나면 차원이 하나 줄고, 슬라이싱이면 차원이 유지된다.

문법 `배열[:, 열번호]` 열 전체 꺼내기 — 실무 최빈출

정의 콜론으로 모든 행을 지정하고, 심표 뒤에 열 번호를 쓴다. 그 열의 값이 세로로 전부 모여 나온다.

형태 `배열[:, 열번호]`

```
data = np.array([
    [70, 2.1],
    [72, 2.3]
])

print(data[:, 0])    # 0열 전체 (온도)
print(data[:, 1])    # 1열 전체 (진동)
```

```
▶ [70. 72.]
▶ [2.1 2.3]
```

왜 쓰나 설비 분석에서 가장 잦은 작업이다. 열 하나가 센서 하나이므로, `data[:, 0]`은 "온도 센서의 전체 시점값"이 된다. 이 배열이 나와야 평균·최댓값·표준편차 같은 통계를 낼 수 있다.

응용 행은 번호만으로 뽑히는데 열은 왜 콜론이 필요한가? **행이 배열의 기본 단위이기 때문이다.** 행은 이미 한 덩어리로 묶여 있지만, 열은 모든 행을 가로질러 값을 하나씩 모아야 한다. 그래서 "모든 행에서"라는 지시가 반드시 앞에 붙는다.

```
data = np.array([
    [1151, 42.8],
    [1408, 46.3],
    [2861, 4.6],
    [1410, 65.7]
])

rpm      = data[:, 0]    # 회전수 열
torque   = data[:, 1]    # 토크 열

print("회전수:", rpm)
print("토크 :", torque)
print("최대 회전수:", rpm.max())
```

```
▶ 회전수: [1151. 1408. 2861. 1410.]
▶ 토크 : [42.8 46.3  4.6 65.7]
▶ 최대 회전수: 2861.0
```

자주 하는 실수

`data[:, 0]`으로 쓰면 안 된다. 그건 "전체를 꺼낸 뒤 0행"이라 `data[0]`과 같아진다. **반드시 대괄호 하나 안에**
선표로 — `data[:, 0]`.

7-4. 행 추출과 열 추출 — 왜 방식이 다른가

구분	표현	꺼내는 것	설비 데이터 의미
행 추출	<code>data[0]</code>	그 행의 모든 값	그 시점의 모든 센서값
열 추출	<code>data[:, 0]</code>	모든 행의 그 열 값	그 센서의 모든 시점값
값 하나	<code>data[0, 1]</code>	특정 행·열의 값 하나	특정 시점의 특정 센서값
부분 표	<code>data[0:2, 0:1]</code>	행·열 둘 다 잘라낸 표	앞 2시점 × 첫 센서

7-5. 인덱싱·슬라이싱 총정리

목적	표현	결과	차원
값 하나	<code>a[2]</code>	2번 값	스칼라
구간	<code>a[1:4]</code>	1·2·3번	1차원
간격	<code>a[::2]</code>	0·2·4번	1차원
뒤집기	<code>a[::-1]</code>	거꾸로 전체	1차원
2차원 값 하나	<code>data[0, 1]</code>	0행 1열 값	스칼라
행 전체	<code>data[0]</code>	0행 모두	1차원
열 전체	<code>data[:, 0]</code>	0열 모두	1차원
부분 표	<code>data[0:2, :]</code>	0·1행 전체	2차원

설비 적용

오늘 배운 마지막 표현 `data[:, 0]`이 다음 시간 전체의 출발점이다. 열 하나를 뽑아야 `np.mean()`으로 평균을 내고, `data[:, 0] > 2000` 같은 조건을 걸어 이상값을 골라낼 수 있다. 인덱싱·슬라이싱은 그 자체로는 밋밋해 보이지만, 모든 분석이 여기서 시작한다.

치트시트

오늘 나온 전부 — 한 장 요약

A-1. 터미널 명령 (파이썬 파일이 아니라 검은 창)

명령	하는 일	비고
<code>python -m venv .venv</code>	현재 폴더에 가상환경 생성	폴더명은 관례상 .venv
<code>source .venv/bin/activate</code>	가상환경 활성화 (mac/Linux)	프롬프트에 (.venv) 표시
<code>.venv\Scripts\activate</code>	가상환경 활성화 (Windows)	PowerShell은 Activate.ps1
<code>deactivate</code>	가상환경 종료	작업이 끝나면
<code>pip install numpy</code>	라이브러리 설치	활성화 후에 실행할 것
<code>pip list</code>	설치된 목록 확인	버전까지 표시
<code>pip freeze > requirements.txt</code>	설치 목록을 파일로 고정	팀 공유·재현용

A-2. 배열 생성 함수

함수	언제 쓰나	핵심 규칙	예
<code>np.array(리스트)</code>	값을 이미 알 때	소수 하나면 전체 float	<code>np.array([70, 72])</code>
<code>np.arange(시작, 끝, 간격)</code>	간격이 중요할 때	끝값 제외	<code>np.arange(0, 10, 2)</code>
<code>np.linspace(시작, 끝, 개수)</code>	개수로 나눌 때	끝값 포함, 항상 float	<code>np.linspace(0, 1, 5)</code>
<code>np.zeros(n)</code>	빈 그릇이 필요할 때	기본 float, 2차원은 튜플	<code>np.zeros((2, 3))</code>
<code>np.ones(n)</code>	1로 채울 때	비율·가중치 출발점	<code>np.ones(4)</code>
<code>np.full(n, 값)</code>	특정 값으로 채울 때	채울 값이 dtype 결정	<code>np.full(4, 7)</code>

A-3. 속성(괄호 없음)과 메서드(괄호 있음)

이름	분류	알려주는 것 / 하는 일	기억할 점
<code>.ndim</code>	속성	차원 개수	shape의 숫자 개수와 같다
<code>.shape</code>	속성	행·열 크기 (튜플)	앞이 행, 뒤가 열
<code>.size</code>	속성	값 총 개수	shape을 곱한 값
<code>.dtype</code>	속성	자료형 (int64 / float64)	모양과 무관한 독립 정보
<code>.astype(형)</code>	메서드	자료형 변환	반올림이 아니라 버림
<code>.reshape(행, 열)</code>	메서드	모양 변경	size가 같아야 함, -1은 자동
<code>.flatten()</code>	메서드	1차원으로 펴기	책 읽는 방향으로

개념

세 메서드 `astype` · `reshape` · `flatten`은 전부 원본을 바꾸지 않고 새 배열을 돌려준다. 결과를 새 이름에 담지 않으면 아무 일도 일어나지 않는다. 이것이 오늘 가장 놓치기 쉬운 규칙이다.

A-4. 값 추출 문법

표현	뜻	결과 차원
<code>a[0]</code>	0번 값	스칼라
<code>a[-1]</code>	맨 마지막 값	스칼라
<code>a[1:4]</code>	1·2·3번 (4 제외)	1차원
<code>a[:3] / a[3:]</code>	처음부터 / 끝까지	1차원
<code>a[:, :2]</code>	두 칸씩 건너뛰기	1차원
<code>a[:, :-1]</code>	거꾸로	1차원
<code>data[0, 1]</code>	0행 1열 값	스칼라
<code>data[0]</code>	0행 전체	1차원
<code>data[:, 0]</code>	모든 행의 0열 (열 추출)	1차원
<code>data[0:2, :]</code>	0·1행 전체	2차원

오류·함정 사전

x 이렇게 하면 → 무슨 일이 → ✓ 바로잡기

B-1. 오늘 만나기 쉬운 오류 10가지

x 이렇게 하면	무슨 일이 일어나나	✓ 바로잡기
<code>pip install numpy</code> (가상환경 없이)	<code>error: externally-managed-environment</code>	<code>python -m venv .venv</code> 후 활성화하고 설치
<code>np.array(1, 2, 3)</code>	<code>TypeError — 인자를 셋으로 받음</code>	<code>np.array([1, 2, 3])</code> 리스트로 묶기
<code>np.arange(5)</code> 를 6개로 착각	실제는 0~4, 다섯 개	끝값 제외 — <code>np.arange(6)</code> 이 0~5
<code>np.linspace(0,10,2)</code> 를 "2 간격"으로	간격이 아니라 개수 — <code>[0. 10.]</code>	간격을 정하려면 <code>np.arange(0,10,2)</code>
<code>np.zeros(2, 3)</code>	3을 dtype으로 해석 → 오류	<code>np.zeros((2, 3))</code> 괄호 한 겹 더
<code>arr.shape()</code>	<code>TypeError: 'tuple' is not callable</code>	<code>arr.shape</code> 속성은 괄호 없음
<code>arr.flatten</code> (괄호 누락)	오류 없이 메서드 객체가 찍힘 — 더 위험	<code>arr.flatten()</code> 메서드는 괄호 필수
<code>a.astype(int)</code> 만 실행	결과가 사라짐. 원본 a는 그대로	<code>z = a.astype(int)</code> 새 이름에 담기
<code>a.reshape(3, 5)</code> (size 12)	<code>ValueError: cannot reshape</code>	행×열 = size 확인. <code>reshape(-1, 4)</code>
<code>data[:,0]</code>	전체를 꺼낸 뒤 0행 → 행이 나옴	<code>data[:, 0]</code> 대괄호 하나에 심표

B-2. 제출 코드 교정 — 실제로 실행해 확인한 것

Practice/10_01_example.py와 Practice/10_02_example.py, Python/21_numpy.py,

Python/22_numpy_indexing_slicing.py를 실제로 돌려 보고 정리했다. 오류가 나지 않고 결과만 틀리는 버그가 하나 있었는데, 이런 종류는 눈으로만 보면 절대 못 잡는다.

① 실습 1 — 화씨 변환 공식 오류 (결과가 틀림)

Practice/10_01_example.py

```
# x 제출 코드
temp_Fahrenheit = (temp_celsius * 9 / 5 + 3).round(1)

# ✓ 올바른 공식
temp_Fahrenheit = (temp_celsius * 9 / 5 + 32).round(1)
```

실행해서 비교

```
c = np.array([56.5, 25.2, -3.1])

print("+3 :", (c * 9 / 5 + 3).round(1))
print("+32 :", (c * 9 / 5 + 32).round(1))
```

```
▶ +3 : [104.7  48.4  -2.6]   ← 제출 코드의 결과
▶ +32 : [133.7  77.4  26.4]   ← 정답
```

자주 하는 실수

화씨 변환은 $^{\circ}\text{F} = ^{\circ}\text{C} \times 9/5 + 32$ 다. +3은 오타지만 파이썬은 아무 불평도 하지 않는다. 56.5도가 104.7°F로 나와도 프로그램은 정상 종료된다. 실제 정답은 133.7°F — 29도나 차이 난다. 설비 임계치 판정에 이 값을 쓰면 정상을 이상으로, 이상을 정상으로 뒤집는다. 상수는 코드에 직접 박지 말고 이름을 붙여 두면 이런 오타가 눈에 띈다.

권장 — 이름을 붙이고 계산까지

```
# 상수에 이름을 붙이면 검산이 쉬워진다
C_TO_F_RATIO = 9 / 5
C_TO_F_OFFSET = 32

temp_f = (temp_c * C_TO_F_RATIO + C_TO_F_OFFSET).round(1)

# 검산: 0도는 32도, 100도는 212도
print(np.array([0.0, 100.0]) * C_TO_F_RATIO + C_TO_F_OFFSET)
```

```
▶ [ 32. 212.]
```

② 실습 4 — shape·size 출력 누락

교안 실습 4의 요구는 "ndim으로 차원, shape으로 형태, size로 전체 개수 확인 → 세 속성값 출력"이다. 제출 코드는 ndim만 출력했다.

Practice/10_01_example.py 실습 4

```
# x 제출 코드
print(f"차원(ndim): {equip_data.ndim}")

# ✓ 세 속성 모두
print(f"차원(ndim) : {equip_data.ndim}")
print(f"형태(shape): {equip_data.shape}")
print(f"개수(size) : {equip_data.size}")
print(f"자료형 : {equip_data.dtype}")
```

▶ 차원(ndim) : 2
▶ 형태(shape): (2, 3)
▶ 개수(size) : 6
▶ 자료형 : int64

③ 실습 2 — 쓰지 않는 변수

Practice/10_01_example.py 실습 2

```
# x length 를 만들었지만 아래에서 쓰지 않는다
length = random.randint(1, 20)
np_linspace = np.linspace(0, 100, 20)

# ✓ 만든 변수를 실제로 쓴다
length = random.randint(2, 20) # linspace 개수는 2 이상이어야 의미가 있다
np_linspace = np.linspace(0, 100, length)
print(f"0~100 을 {length}개로 균등 분할: {np_linspace}")
```

▶ 0~100 을 5개로 균등 분할: [0. 25. 50. 75. 100.]

주의

쓰지 않는 변수는 오류가 아니지만 읽는 사람에게 "이 값이 아래에서 쓰이겠구나" 하는 잘못된 기대를 준다. 코드 리뷰에서 가장 먼저 지적당하는 항목이고, 포트폴리오 레포에 남아 있으면 완성도가 낮아 보인다. 안 쓸 거면 지우고, 쓸 거면 실제로 쓰자.

④ 22_numpy_indexing_slicing.py — len() 대신 .shape

Python/22_numpy_indexing_slicing.py

```
# ✕ 리스트 사고방식
row = len(temp_2d_rand)
col = len(temp_2d_rand[0])

# ✓ 배열 사고방식 — 한 줄로 끝난다
row, col = temp_2d_rand.shape
print(f"행 {row} · 열 {col}")
```

▶ 행 3 · 열 10

len()도 동작하기는 한다. 하지만 2차원에서 len()은 **행의 개수만** 알려주고, 3차원 이상에서는 첫 축 크기만 준다.

.shape은 모든 차원을 한 번에 돌려주므로 배열에서는 항상 .shape을 쓰는 것이 정석이다. 강사가 "배열답게 쓰라"고 강조한 지점이 이런 부분이다.

표현	2차원에서 결과	3차원에서 결과	권장
len(arr)	행 개수만 (2)	첫 축만 (10)	✕
arr.shape	(2, 3) 전체	(10, 2, 3) 전체	✓
arr.size	6 (전체 칸)	60 (전체 칸)	✓

⑤ 21_numpy.py — 주석 오타

✕ 원문	✓ 바로잡기
파이썬 라이브러리 관리 매니저인 pip	파이썬 라이브러리 관리 매니저인 pip
위 intr값들의 리스트를 지니하게 정하라	위 int 값들의 리스트를 배열로 정하라
3. [작업 / 실행 끝나고) 가상환경 종료]	3. 작업·실행이 끝나면 가상환경 종료

연결

주석 오타는 실행에 영향이 없지만, **GitHub에 올라간 코드는 면접관이 읽는다**. 깨진 주석이 섞여 있으면 "급하게 따라 친 코드"로 보인다. 회차 정리할 때 주석까지 다듬어 커밋하는 습관이 포트폴리오 품질을 만든다.

실습 하이라이트

제출한 것 · 빠진 것 · 다음 시간

C-1. 오늘 나온 실습 전체

실습	교안 요구	제출 여부	한 줄 평
10_01 실습 1	섭씨 배열 → 화씨 변환	제출	공식 오류 (+3 → +32)
10_01 실습 2	linspace로 균등 분할	제출	length 변수 미사용
10_01 실습 3	arange로 시간축, 간격 바꿔 관찰	제출	간격 2·0.5 두 번 비교 — 좋음
10_01 실습 4	ndim·shape·size 확인	제출	ndim만 출력
10_01 실습 5	dtype 확인 후 astype 변환	미제출	아래 보충답안
10_01 실습 6	reshape로 표 변환	제출	reshape(-1, 4) 활용 — 좋음
10_01 실습 7	한 줄 → 시점×센서 표	미제출	아래 보충답안
10_01 실습 8	생성→확인→변환 한 흐름	미제출	아래 보충답안
10_02 실습 1	인덱싱·슬라이싱으로 추출	미제출	수업 중 데이터만 제시
10_02 실습 2	행·열 단위 추출	데이터만	추출 코드 없음

C-2. 미작성 실습 보충답안

10_01 실습 5 — 자료형 확인과 변환하기

실습 5 보충답안

```
import numpy as np

# 소수점이 있는 측정값 배열
measure = np.array([70.5, 69.8, 73.7, 99.9])

print(f"원본   : {measure}")
print(f"자료형  : {measure.dtype}")

# astype 으로 정수 변환 (버림에 주의)
measure_int = measure.astype(int)
print(f"정수 변환 : {measure_int}")
print(f"변환 후   : {measure_int.dtype}")
print(f"원본 유지 : {measure}")
```

- ▶ 원본 : [70.5 69.8 73.7 99.9]
- ▶ 자료형 : float64
- ▶ 정수 변환 : [70 69 73 99]
- ▶ 변환 후 : int64
- ▶ 원본 유지 : [70.5 69.8 73.7 99.9]

10_01 실습 7 — 센서 데이터 표로 정리하기

실습 7 보충답안

```
import numpy as np

TIMES, SENSORS = 3, 2    # 시점 3개, 센서 2개

# 시점 × 센서 개수만큼 연속값 생성
raw = np.arange(TIMES * SENSORS)
print(f"한 줄 데이터: {raw} (size {raw.size})")

# 행=시점, 열=센서 로 정리
table = raw.reshape(TIMES, SENSORS)
print(table)
print(f"정리 후 shape: {table.shape}")
```

```
▶ 한 줄 데이터: [0 1 2 3 4 5] (size 6)
▶ [[0 1]
   [2 3]
   [4 5]]
▶ 정리 후 shape: (3, 2)
```

10_01 실습 8 — 배열 생성부터 정리까지

실습 8 보충답안 — 오늘 배운 것 전부

```
import numpy as np

# ① 생성 — 시점 3개 × 센서 2개(온도, 진동)가 한 줄로 들어옴
sensor = np.array([70.5, 2.1, 69.8, 2.3, 73.7, 2.0])

# ② 구조 확인
print(f"shape : {sensor.shape}")
print(f"dtype : {sensor.dtype}")
print(f"size : {sensor.size}")

# ③ 분석용 표로 정리
table = sensor.reshape(-1, 2)
print(table)
print(f"정리 후 shape: {table.shape}")

# ④ 열 추출까지 — 오늘 배운 것 전부 이어 붙이기
print(f"온도 열: {table[:, 0]}")
print(f"진동 열: {table[:, 1]}")
```

```
▶ shape : (6,)
▶ dtype : float64
▶ size : 6
▶ [[70.5  2.1]
▶  [69.8  2.3]
▶  [73.7  2. ]]
▶ 정리 후 shape: (3, 2)
▶ 온도 열: [70.5 69.8 73.7]
▶ 진동 열: [2.1 2.3 2. ]
```

10_02 실습 1 — 특정 센서·구간 추출하기

실습 1 보충답안

```
import numpy as np
```

```
rpm = np.array([1551, 1408, 1498, 1443, 1425, 1558, 2861, 1410])
print(f"회전수 배열: {rpm}")
```

```
# 인덱싱 — 첫 시점, 마지막 시점
print(f"첫 시점 : {rpm[0]}")
print(f"마지막 시점: {rpm[-1]}")
```

```
# 슬라이싱 — 앞 구간, 두 칸 간격
print(f"앞 4개 : {rpm[:4]}")
print(f"두 칸 간격: {rpm[::2]}")
print(f"마지막 3개: {rpm[-3:]}")
```

```
▶ 회전수 배열: [1551 1408 1498 1443 1425 1558 2861 1410]
▶ 첫 시점 : 1551
▶ 마지막 시점: 1410
▶ 앞 4개 : [1551 1408 1498 1443]
▶ 두 칸 간격: [1551 1498 1425 2861]
▶ 마지막 3개: [1558 2861 1410]
```

10_02 실습 2 — 행·열 단위로 추출하기

실습 2 보충답안

```
import numpy as np

# 행=설비, 열=(회전수, 토크)
data = np.array([
    [1151, 42.8],
    [1408, 46.3],
    [2861, 4.6],
    [1410, 65.7]
])

# 행 추출 — 특정 설비의 값 전체
print(f"설비 3호(2행): {data[2]}")

# 열 추출 — 모든 설비의 회전수 / 토크
print(f"회전수 열: {data[:, 0]}")
print(f"토크 열 : {data[:, 1]}")

# 값 하나
print(f"설비 3호 토크: {data[2, 1]}")
```

- ▶ 설비 3호(2행): [2861. 4.6]
- ▶ 회전수 열: [1151. 1408. 2861. 1410.]
- ▶ 토크 열 : [42.8 46.3 4.6 65.7]
- ▶ 설비 3호 토크: 4.6

C-3. 다음 시간 예고 — 진도 밖이라 뺀 것

오늘은 교안 10_02의 **PART 01(인덱싱과 슬라이싱, p4~p16)**까지만 나갔다. 아래 두 파트는 진도 밖이라 이 책에서 다루지 않았다. 다만 오늘 배운 `data[:, 0]`이 그대로 이어지는 내용이라, 예고만 짚어 둔다.

다음 파트	주제	핵심 표현	오늘과의 연결
PART 02	배열 산술 연산 · 스칼라 연산	<code>a + b · a * 2</code>	리스트 곱셈 문제의 진짜 해답
PART 02	브로드캐스팅	<code>data - base</code>	모양이 달라도 계산되는 원리
PART 02	비교 연산과 불리언 배열	<code>a > 2000</code>	열을 뽑아야 조건을 걸 수 있다
PART 02	Boolean indexing	<code>a[a > 2000]</code>	이상 징후 탐색의 기본 도구

다음 파트	주제	핵심 표현	오늘과의 연결
PART 02	조건 처리	<code>np.where(조건, 참, 거짓)</code>	골라낼 것인가, 표시할 것인가
PART 02	다중 조건 결합	<code>(a>10) & (b<5)</code>	괄호 누락이 최다 실수
PART 03	합계·평균	<code>np.sum · np.mean</code>	열 추출 후 첫 통계
PART 03	중앙값·최대·최소	<code>np.median · max</code>	평균의 약점을 보완
PART 03	분산·표준편차	<code>np.var · np.std</code>	이상 탐지의 기초
PART 03	axis (행·열 방향)	<code>np.mean(data, axis=0)</code>	센서별 통계가 가장 잦다

연결

강사가 수업 중 길게 이야기한 **정규분포와 이상치 제거**가 PART 03의 배경이다. "국회의원 재산 평균" 예시로 설명했듯, 극단값 하나가 평균 전체를 왜곡한다. 그래서 평균만 보지 않고 **중앙값·표준편차를 함께** 보고, 정규분포를 크게 벗어난 값은 **클렌징(cleansing)**으로 걸러낸 뒤 통계를 낸다. 오늘 배운 열 추출이 그 모든 작업의 첫 단계다.

주의

배포된 보조 교재 네 개(`10_01_numpy_guide`, `10_01_numpy_math_guide`, `10_02_numpy_part2_guide`, `10_02_numpy_part2_math_guide`)에는 오늘 안 나간 PART 02·03 내용까지 들어 있다. 특히 `part2_math_guide`의 **3-시그마(3σ) 규칙**은 예지보전의 핵심 개념이니, 다음 시간 전에 미리 읽어 두면 수업이 훨씬 쉬워진다.

오늘 넘은 선 — 리스트에서 배열로

9주 동안 배운 것은 전부 **파이썬 기본 문법**이었다. 오늘은 처음으로 그 밖으로 나갔다. 터미널을 열어 가상환경을 만들고, 외부 라이브러리를 설치하고, `import numpy as np`로 불러왔다. 이 세 줄이 데이터 분석가의 작업 시작 의식이다.

내용 면에서 오늘 넘은 가장 큰 선은 **"값 하나하나를 반복문으로 돌리는 사고"에서 "배열 전체를 한 번에 다루는 사고"로의 전환**이다. `temp + 5` 한 줄로 모든 값이 바뀌는 장면을 봤다면 절반은 온 것이다. 나머지 절반 — 배열끼리 더하고, 조건을 걸어 골라내고, 통계를 내는 것 — 은 다음 시간에 완성된다.

오늘 할 수 있게 된 것	확인 질문	못 하면 돌아갈 곳
가상환경을 만들고 라이브러리를 설치한다	venv를 왜 만드는지 한 문장으로 설명할 수 있나?	STEP 1
리스트와 배열의 차이를 설명한다	<code>[1,2] * 3</code> 과 <code>np.array([1,2]) * 3</code> 의 차이는?	STEP 2
목적에 맞는 생성 함수를 고른다	간격이 중요하면? 개수가 중요하면?	STEP 3
새 데이터의 구조를 네 속성으로 파악한다	shape과 size의 관계는?	STEP 4
한 줄 데이터를 표로 정리한다	<code>reshape(-1, 3)</code> 의 -1은 무슨 뜻인가?	STEP 5
2차원에서 값 하나를 꺼낸다	<code>data[0, 1]</code> 에서 앞이 행인가 열인가?	STEP 6
특정 센서의 전체 시점값을 뽑는다	<code>data[:, 0]</code> 의 콜론은 무슨 뜻인가?	STEP 7

복습 루틴 — 30분이면 충분하다

오늘 내용은 **손이 기억해야 하는 것**이 대부분이다. 읽는 것만으로는 절대 안 붙는다. 강사도 "헛갈릴 땐 그냥 찍어 보라"고 했다. 아래 순서로 딱 30분만 돌려 보자.

1. 부록 A
5분 · 훑기

2. 실습 5·7·8
10분 · 직접

3. 실습 1 수정
5분 · +32

4. 부록 B
10분 · 소리내어

단계	무엇을	왜
① 부록 A 훑기 (5분)	치트시트를 보며 이름과 역할만 대응시킨다	전체 지도를 머리에 올린다
② 보충답안 직접 치기 (10분)	C-2의 실습 5·7·8을 보지 않고 쳐 본다	손이 기억하게 만든다
③ 실습 1 고치기 (5분)	+3 을 +32 로 고치고 0도·100도로 검산	내가 낸 버그를 내 손으로 잡는다
④ 부록 B 소리내어 읽기 (10분)	$x \rightarrow \checkmark$ 를 입으로 말하며 넘긴다	같은 실수를 두 번 하지 않는다

설비 적용

취업 관점에서 오늘 내용의 값어치는 분명하다. **제조 DX·AX 직군 코딩 테스트와 실무 면접에서 넘파이는 사실상 기본 소양**이다. 특히 `shape`을 읽고 데이터 구조를 설명하는 능력, `reshape`으로 로그를 표로 세우는 능력, `data[:, 0]`으로 센서 하나를 뽑아내는 능력은 그대로 질문이 된다. 오늘 만든 보충답안 코드를 GitHub 레포에 정리해 커밋해 두면, 나중에 "넘파이 다뤄 봤나요?"에 링크로 답할 수 있다.

다음 시간에는 오늘 뽑아낸 열 하나에 조건을 걸어 이상값을 골라내고, 평균과 표준편차를 내 본다. **분석다운 분석이 그때부터 시작된다.**

— 끝 —